

ДЕРЖАВНИЙ УНІВЕРСИТЕТ
ІНФОРМАЦІЙНО-КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА КОМП'ЮТЕРНИХ НАУК

КВАЛІФІКАЦІЙНА РОБОТА

на тему:

**“Розробка програмної системи для візуалізації, порівняння та аналізу
ефективності алгоритмів сортування засобами Java”**

на здобуття освітнього ступеня бакалавра

зі спеціальності ЕЗ – Комп'ютерні науки
(код, найменування спеціальності)

освітньо-професійної програми Комп'ютерні науки
(назва)

Кваліфікаційна робота містить результати власних досліджень. Використання
ідей, результатів і текстів інших авторів мають посилання на відповідне джерело

(підпис)

Богдан ЯРОВИЙ
(Ім'я, ПРІЗВИЩЕ здобувача)

Виконав: здобувач вищої освіти
Богдан ЯРОВИЙ

група КНД-42

Керівник: Юлія БЕРЕЗОВСЬКА

науковий ступінь, доктор філософії
вчене звання

Рецензент:

(Ім'я, ПРІЗВИЩЕ)

Київ 2026

ДЕРЖАВНИЙ УНІВЕРСИТЕТ
ІНФОРМАЦІЙНО-КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ
Навчально-науковий інститут інформаційних технологій

Кафедра: Комп'ютерних наук

Ступінь вищої освіти: Бакалавр

Спеціальність: F3 Комп'ютерні науки

Освітньо-професійна програма: Комп'ютерні науки

ЗАТВЕРДЖУЮ

Завідувач кафедри комп'ютерних наук

_____ Віктор ВИШНІВСЬКИЙ

« _____ » _____ 2026 р.

**ЗАВДАННЯ
НА КВАЛІФІКАЦІЙНУ РОБОТУ**

_____ Яровому Богдану Сергійовичу

(прізвище, ім'я, по батькові здобувача)

1. Тема кваліфікаційної роботи: Розробка програмної системи для візуалізації, порівняння та аналізу ефективності алгоритмів сортування засобами Java, керівник кваліфікаційної роботи: Юлія БЕРЕЗОВСЬКА доктор філософії, затверджені наказом Державного університету інформаційно-комунікаційних технологій від «20» лютого 2026 р. №51

2. Строк подання кваліфікаційної роботи: «29» травня 2026.

3. Вихідні дані до кваліфікаційної роботи:

- a. Науково-технічна література з теорії алгоритмів;
- b. Вихідний код програмної системи SortBench, реалізованої засобами Java;
- c. Документація Java SE (Java Swing, Java2D API).

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити)

- a. Дослідження теоретичних основ алгоритмів сортування та методів оцінки їх ефективності
- b. Проєктування та розробка програмної системи для візуалізації алгоритмів сортування
- c. Тестування системи та порівняльний аналіз ефективності алгоритмів

5. Перелік графічного матеріалу: презентація

- a. Актуальність теми та мета роботи
- b. Огляд існуючих рішень для візуалізації алгоритмів
- c. Архітектура програмної системи SortBench
- d. Механізм record-and-replay
- e. Демонстрація роботи системи
- f. Порівняльний аналіз ефективності алгоритмів
- g. Висновки

6. Дата видачі завдання «23» лютого 2026 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів кваліфікаційної роботи	Строк виконання етапів роботи	Примітка
1	Аналіз науково-технічної літератури з теорії алгоритмів сортування	25.02.26 - 10.03.26	виконано
2	Дослідження існуючих рішень для візуалізації алгоритмів	10.03.26 – 20.03.26	виконано
3	Планування та постановка завдання щодо створення системи візуалізації та створення алгоритмів сортування	22.03.26 – 23.03.26	виконано
4	Проектування архітектури програмної системи	23.03.26 – 05.04.26	виконано
5	Реалізація алгоритмів сортування та механізму record-and-replay	05.04.26 – 25.04.26	виконано
6	Розробка графічного інтерфейсу засобами Java Swing	25.04.26 – 03.05.26	виконано
7	Розробка абстракції алгоритму, задля імплементації функціоналу додавання власних реалізацій алгоритмів сортування	03.05.26 – 12.05.26	виконано
8	Тестування та порівняльний аналіз ефективності алгоритмів	12.05.26 – 15.05.26	виконано
9	Оформлення роботи: вступ, висновки, реферат	15.05.26 – 22.05.26	виконано
10	Розробка презентації для захисту	22.05.26 – 25.05.26	виконано

Здобувач вищої освіти

(підпис)

Богдан ЯРОВИЙ
(Ім'я, ПРІЗВИЩЕ)

Керівник
кваліфікаційної роботи

(підпис)

Юлія БЕРЕЗОВСЬКА
(Ім'я, ПРІЗВИЩЕ)

РЕФЕРАТ

Текстова частина кваліфікаційної роботи на здобуття освітнього ступеня бакалавра: 59 стор., 26 рис., 2 табл., 19 джерел.

Мета роботи: розробка програмної системи для візуалізації, покрокового порівняння та аналізу ефективності алгоритмів сортування засобами мови програмування Java.

Об'єкт дослідження: процес візуалізації та порівняння алгоритмів сортування.

Предмет дослідження: методи та засоби розробки інтерактивних програмних систем для навчально-аналітичних цілей на платформі Java.

Короткий зміст роботи: Досліджено основний принцип порівняння даних між собою та використання цього принципу для сортування масивів таких даних.

Розглянуто найпопулярніші алгоритми сортування даних. До прикладу: Bubble Sort, Quick Sort, Merge Sort, Selection Sort.

Проаналізовано наявні у відкритому доступі релевантні рішення та виявлені головні їх недоліки.

Розроблено власне рішення у вигляді графічного застосунку, що дозволяє швидко створювати власні порівняльні алгоритми сортування за допомогою легкої бібліотеки. Графічний застосунок дозволяє візуалізовувати роботу алгоритму сортування та має базові модулі налаштування візуалізації. Бібліотека являє собою легку абстракцію, що дозволяє зручно створювати власні реалізації алгоритмів. Бібліотека розташована в мережі на Maven Central Repository, що дозволяє легко підключити її в код.

КЛЮЧОВІ СЛОВА: АЛГОРИТМИ СОРТУВАННЯ, ВІЗУАЛІЗАЦІЯ, JAVA, JAVA SWING, АНАЛІЗ ЕФЕКТИВНОСТІ, ПРОГРАМНА СИСТЕМА, ПОРІВНЯННЯ АЛГОРИТМІВ, СКЛАДНІСТЬ АЛГОРИТМІВ

ЗМІСТ

ВСТУП.....	9
1 ТЕОРЕТИЧНІ ОСНОВИ АЛГОРИТМІВ СОРТУВАННЯ	13
1.1 Принципи порівняння та впорядкування даних	13
1.2 Класифікація та огляд алгоритмів сортування.....	15
1.3 Аналіз часової та просторової складності	20
1.4 Огляд існуючих рішень для візуалізації алгоритмів	21
2 ПРОЄКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ SORTBENCH. 28	
2.1 Огляд SortBench.....	28
2.2 Визначення вимог та планування MVP	31
2.3 Архітектура системи	34
2.4 Розробка застосунку.....	42
2.5 Розробка бібліотеки SortBenchAbstraction.....	55
2.6 Публікація бібліотеки на Maven Central Repository	58
3 ТЕСТУВАННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ.....	60
3.1 Тестування програмної системи	60
3.2 Обмеження, недоліки та перспективи розвитку SortBench	66
ВИСНОВКИ.....	68
ПЕРЕЛІК ПОСИЛАНЬ	69
ДОДАТКИ.....	71
ПРЕЗЕНТАЦІЯ.....	72

ВСТУП

Актуальність теми: У сучасному світі існує безліч різноманітних інформаційних систем, що постійно маніпулюють петабайтами даних. Для ефективної маніпуляції цими даними використовують різні підходи та алгоритми. Фундаментальними алгоритмами в таких системах є алгоритми сортування. Вони дозволяють не тільки швидко сортувати дані, а і ефективно шукати специфічні дані у великих масивах інформації. Хоча алгоритми сортування є для нас чимось звичним, існують багато складних підходів до сортування даних. Такі алгоритми є дуже ефективними але їх важко зрозуміти через їх складність. Вивчення алгоритмів сортування потребує сильних навичок абстрактного мислення, математичні здібності та високий рівень когнітивної складності.

Окрім вивчення існуючих алгоритмів також постійно йде процес пошуку нових більш ефективних підходів сортування даних. Такий процес потребує значно вищого рівня інтелектуальних здібностей, а також глибоке розуміння багатьох вже існуючих рішень.

Для прискорення вивчення та розробки нових алгоритмів сортування використовують різноманітні інструменти для візуалізації роботи процесу сортування. Такі інструменти дозволяють наочно побачити весь процес покроково, та з набагато меншими зусиллями усвідомити принцип роботи.

Оскільки алгоритми сортування є невід'ємною частиною фундаментальної підготовки програмістів, розвиток якісних навчальних інструментів для їх вивчення є важливим для підвищення рівня українських ІТ-фахівців та конкурентоспроможності вітчизняної ІТ-галузі загалом. [1]

Аналіз останніх досліджень і публікацій: У відкритому доступі існує чимало цікавих рішень візуалізації алгоритмів сортування. Серед них можна виділити найпопулярніші:

1. Visualgo - web-ресурс, на якому можна подивитись роботу попередньо визначених алгоритмів сортування; [2]

2. Toptal Sorting Algorithms Animations - web-ресурс, що дозволяє побачити одночасну роботу попередньо визначених алгоритмів сортування; [3]
3. Algorithm Visualizer - web-ресурс, на якому можна розширено переглянути роботу різних алгоритмів сортування, та навіть реалізувати свій алгоритм на декількох мовах програмування. Помітним недоліком, є необхідність розуміння утиліт ресурсу та необхідність ручного визначення візуалізації. Тобто, користувач має самостійно реалізувати алгоритм та визначити як він має візуалізуватись на екрані. [4]
4. The Sound of Sorting - настільний застосунок, що дозволяє зручно переглянути попередньо визначені алгоритми сортування. [5]
5. USFCA Data Structure Visualizations - web-ресурс, що дозволяє побачити роботу попередньо визначених алгоритмів сортування; [6]

Під час аналізу було виявлено головний недолік цих систем. Майже кожен візуалізатор пропонував попередньо визначені алгоритми сортування, де користувачу не було надано можливість розробити власний. Було виявлено лише один ресурс, який дозволяє розробляти власні рішення - Algorithm Visualizer. Algorithm Visualizer пропонує користувачу цілу платформу для візуалізації, на якому користувач може зображати не лише алгоритми сортування, а й візуалізацію проходження графів. Ресурс Algorithm Visualizer пропонує широкий спектр можливостей, але для реалізації алгоритму потребує також активної підтримки допоміжних функцій. Користувач має думати не лише про роботу алгоритму, а й про те коли і як зображати роботу алгоритму. Це збільшує когнітивне навантаження на користувача ресурсом, та призводить до потенційного ризику хибного розуміння реалізованих алгоритмів.

Виявлені недоліки існуючих рішень обумовлюють доцільність розробки програмної системи, яка забезпечує простий механізм інтеграції власних алгоритмів без необхідності ручного визначення логіки візуалізації.

Мета і завдання дослідження: Мета кваліфікаційної роботи - розробка програмної системи для покрокової візуалізації, порівняння та аналізу

ефективності алгоритмів сортування засобами Java, а також забезпечення можливості інтеграції власних реалізацій алгоритмів користувачем.

Для досягнення поставленої мети необхідно вирішити такі завдання:

1. проаналізувати теоретичні засади алгоритмів сортування та методи оцінки їх ефективності;
2. здійснити огляд існуючих рішень для візуалізації алгоритмів;
3. спроектувати архітектуру програмної системи на основі підходу record-and-replay;
4. реалізувати алгоритми сортування та графічний інтерфейс засобами Java Swing;
5. розробити абстракцію для створення власних алгоритмів, та інтегрувати можливість додавання власних рішень в застосунок;
6. провести тестування та порівняльний аналіз реалізованих алгоритмів.

Об'єкт дослідження: процес покрокової візуалізації та порівняння алгоритмів сортування.

Предмет дослідження: методи та засоби розробки інтерактивних програмних систем для навчально-аналітичних цілей на платформі Java.

Методи дослідження: У роботі використано такі методи:

1. теоретичний аналіз науково-технічної літератури - для дослідження принципів побудови алгоритмів сортування та методів оцінки їх ефективності;
2. порівняльний аналіз - для огляду існуючих рішень візуалізації та зіставлення характеристик реалізованих алгоритмів;
3. об'єктно-орієнтоване проєктування - для розробки архітектури програмної системи; тестування програмного забезпечення - для перевірки коректності роботи реалізованих алгоритмів та графічного інтерфейсу.

Наукова новизна та практична значущість: Зважаючи на недоліки вже існуючих систем для відображення роботи алгоритмів сортування, SortBench пропонує користувачу простий спосіб описання власних алгоритмів

використовуючи легковісну бібліотеку SortBenchAbstraction. SortBenchAbstraction надає користувачу інструменти ізольовані від будь-якої логіки візуалізації. Такий підхід дозволяє користувачу сконцентруватись виключно на написанні логіки алгоритму, а про все інше потурбується SortBench автоматично.

Практична цінність отриманих результатів полягає у наданні зручного та простого інструменту для вивчення та розробки алгоритмів сортування. SortBench вимагає від користувача лише необхідний мінімум, що дозволяє сконцентруватись на навчанні або розробці.

1 ТЕОРЕТИЧНІ ОСНОВИ АЛГОРИТМІВ СОРТУВАННЯ

1.1 Принципи порівняння та впорядкування даних

Дані - це інформація. Якщо ми говоримо про дані в комп'ютерних системах, то сучасні комп'ютерні системи зберігають інформацію шляхом запису в двійкові значення. Двійкове представлення інформації можна уявити як "0001 0011", і цей набір двійкових значень представляє число 19. Можна також записати величезне число у двійковому вигляді, де наприклад число 199 216 виглядатиме так "0011 0000 1010 0011 0000" у двійковому представленні. [7]

Ми можемо зберігати не лише числа у двійковому вигляді, але відображати і текст. Наприклад рядок "Hello world!", буде виглядати як "0100 1000, 0110 0101, 0110 1100, 0110 1100, 0110 1111, 0010 0000, 0111 0111, 0110 1111, 0111 0010, 0110 1100, 0110 0100, 0010 0001". Тут кожен символ має своє числове представлення, а число можна вже відобразити у двійковій формі. Це є можливим через існування таблиці UNICODE, що по суті є співставленням чисел до символів. [8]

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43
44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65
66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87
88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109
110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131
132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153
154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197
198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219

Рисунок 1.1 - Таблиця UNICODE

Тобто, використовуючи таблицю UNICODE, можна представити символи у вигляді чисел. Таким чином зберігається текст в комп'ютерних системах, де комп'ютер знає про UNICODE, і в потрібний для нас момент перетворює набори чисел у символічне представлення, хоча текст так і залишається лише числами.

Таблиця 1.1 Декодований вигляд рядка "Hello world!"

Символ	Індекс в таблиці UNICODE	Двійкове представлення
Н	72	0100 1000
e	101	0110 0101
l	108	0110 1100
l	108	0110 1100
o	111	0110 1111
	32	0010 0000
w	119	0111 0111
o	111	0110 1111
r	114	0111 0010
l	108	0110 1100
d	100	0110 0100
!	33	0010 0001

Крім чисел та рядків, сучасні комп'ютерні системи оперують більш складними об'єктами. Сучасні програми мають безліч абстракцій, що дозволяє ефективно обробляти дані. Наприклад у будь-якому онлайн магазині є об'єкти по типу: товар, покупець, постачальник, запис про оплату, співробітники. Такі об'єкти складаються з інших об'єктів та зберігають в собі різні дані. До прикладу візьмемо абстракцію товар, який буде зберігати інформацію про:

1. назву товару - текст;
2. категорію товару - текст;
3. ціну товару - число.

Якщо ми можемо взяти числове представлення ціни товару тому що це і є число, категорії товару тому що це текст який ми можемо перетворити через таблицю UNICODE, назву товару так само через таблицю UNICODE, то додавши всі числові представлення характеристик об'єкта, ми можемо представити складний об'єкт у вигляді числа.

$$\text{числовепредставленняоб'єкта} = \text{назва} + \text{категорія} + \text{ціна} \quad (1.1)$$

Таким чином можна представити абсолютно будь-який об'єкт в комп'ютерних системах з будь-якою складністю в числовому вигляді.

Якщо в нас є два числа, то ми можемо зрозуміти, яке з них більше, просто порівнявши їх. Таким чином, розуміючи, як порівнювати об'єкти будь-якої складності, ми можемо організувати їх в зростаючому порядку, тобто відсортувати. На практиці ж числове представлення об'єктів може бути корисним лише для того, щоб сказати що об'єкти точно різні, адже якщо порівнювати об'єкти за усіма характеристиками одразу, це як спроба визначити який об'єкт кращий, що насправді не можливо. Тому для порівняння об'єктів потрібно визначити за яким параметром об'єкти мають порівнюватись, тобто обрати ключ порівняння. Якщо це ціна товару, то при порівнянні об'єктів, буде враховуватись лише ціна, що дозволить визначити, які товари дорожче. В Java для цього використовують *interface Comparable < T >*, щоб визначити, які ключі мають використовуватись для порівняння об'єктів одного типу. [9]

1.2 Класифікація та огляд алгоритмів сортування

Сортування (англ. sorting) - це процес впорядкування елементів за певними ознаками. Точнішою була б назва впорядкування (англ. ordering), але через переважаність слова «порядок» (англ. order) напрочуд різними значеннями ним не користуються. [10]

Алгоритм сортування – це алгоритм для впорядкування елементів у деякій структурі даних за зростанням чи спаданням. У випадку наявності елементів з однаковими значеннями, в упорядкованій послідовності вони розташовуються поруч один за одним у будь-якому порядку. [11]

Алгоритми сортування діляться на два основних класи:

- порівняльні;
- не порівняльні.

Порівняльні алгоритми сортування є найбільш поширеним класом. Такі алгоритми сортують елементи шляхом порівняння їх між собою. Існує безліч реалізацій порівняльних алгоритмів сортування, і з них можна виділити найбільш відомі:

1. Bubble Sort
2. Quick Sort
3. Select Sort
4. Merge Sort

Головною перевагою порівняльних алгоритмів сортування є їх універсальність. Вони використовуються будь де, і не потребують специфічних умов для роботи. Такі алгоритми можуть працювати з будь-якою кількістю даних.

Не порівняльні алгоритми сортування не порівнюють елементи на пряму, а працюють використовуючи специфіку даних. З таких алгоритмів можна навести декілька відомих:

1. Counting Sort
2. Radix Sort
3. Bucket Sort

Такі алгоритми не можна застосувати будь-де, але їх використовують для сортування специфічних даних. Такі алгоритми можуть сортувати дані значно ефективніше за порівняльні алгоритми, проте вони вимагають певних передумов.

Найпоширенішими алгоритмами сортування є саме порівняльні алгоритми. Основні операції в роботі таких алгоритмів є порівняння(англ. compare) та обмін(англ. swap). Варто розуміти, що не існує єдиного універсального та найефективнішого алгоритму сортування, і кожен алгоритм ефективний для певних задач. Якщо говорити про великі об'єми даних, то найшвидшим алгоритмом є Quick Sort, але він не ефективний при сортуванні маленьких або майже відсортованих об'ємів даних. Усім відомий Bubble Sort є повільним алгоритмом сортування, але він є найпростішим для розуміння, що дозволяє швидко реалізувати просте сортування у власному проекті.

QuickSort — це алгоритм сортування, що базується на принципі "розділяй і володарюй" (divide and conquer). Основна ідея полягає у виборі опорного елемента (pivot) та розподілі масиву на дві частини: елементи менші за pivot розміщуються ліворуч, більші — праворуч. Після цього алгоритм рекурсивно застосовується до кожної з частин.

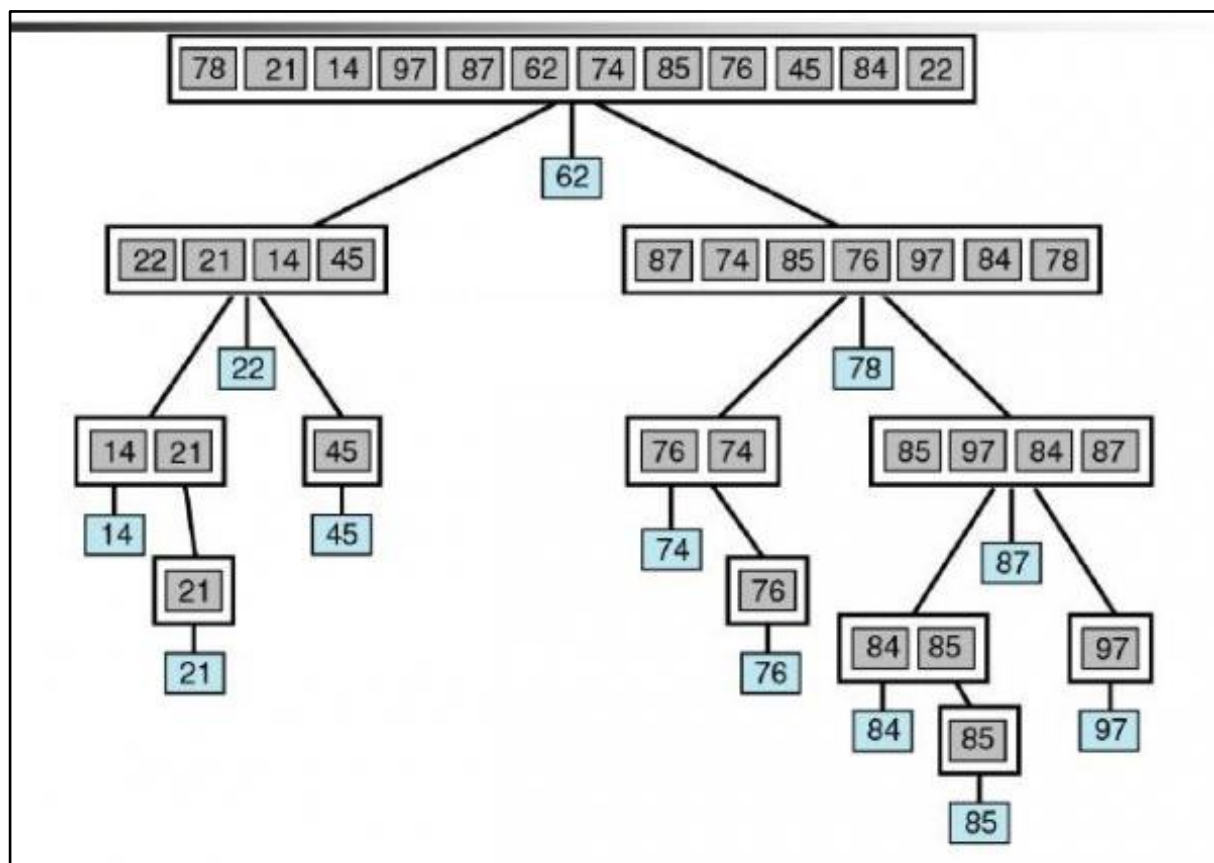


Рисунок 1.2 - Візуалізація роботи алгоритму сортування Quick Sort [12]

Розглянемо роботу алгоритму на прикладі масиву [5, 3, 8, 1, 9, 2]:

1. Обирається опорний елемент, наприклад 5
2. Масив розподіляється: [3, 1, 2] | 5 | [8, 9]
3. Рекурсивно сортується ліва частина [3, 1, 2] і права [8, 9]

Результат: [1, 2, 3, 5, 8, 9]

Часова складність QuickSort у середньому випадку становить $O(n \log n)$. Проте у найгіршому випадку — коли опорний елемент щоразу виявляється найменшим або найбільшим — складність деградує до $O(n^2)$. Це відбувається наприклад на вже відсортованому масиві при виборі першого елемента як pivot.

Для уникнення найгіршого випадку використовують оптимізацію median-of-three — як pivot обирається медіана з трьох елементів: першого, середнього та останнього. Це значно зменшує ймовірність деградації до $O(n^2)$ і саме цей підхід реалізовано в SortBench.

Bubble Sort — це один з найпростіших алгоритмів сортування, що базується на принципі послідовного порівняння сусідніх елементів масиву та їх обміну якщо вони стоять у неправильному порядку. Назва алгоритму походить від того, що більші елементи поступово "спливають" до кінця масиву подібно до бульбашок у воді.

Розглянемо роботу алгоритму на прикладі масиву [5, 3, 8, 1, 2]:

1. Порівнюються перші два елементи: $5 > 3$ — обмін $\rightarrow$ [3, 5, 8, 1, 2]
2. Порівнюються наступні: $5 < 8$ — обміну немає $\rightarrow$ [3, 5, 8, 1, 2]
3. $8 > 1$ — обмін $\rightarrow$ [3, 5, 1, 8, 2]
4. $8 > 2$ — обмін $\rightarrow$ [3, 5, 1, 2, 8]
5. Перший прохід завершено — найбільший елемент 8 стоїть на своєму місці
6. Алгоритм повторює проходи до повного впорядкування масиву

Часова складність Bubble Sort у середньому та найгіршому випадку становить $O(n^2)$, що робить його неефективним на великих масивах. У найкращому випадку — коли масив вже відсортований — складність становить $O(n)$, якщо реалізація містить перевірку на відсутність обмінів за прохід.

Попри низьку ефективність, Bubble Sort має важливе навчальне значення — він є одним з найінтуїтивніших алгоритмів для розуміння базових принципів сортування.

Merge Sort — це алгоритм сортування, що також базується на принципі "розділяй і володарюй". На відміну від QuickSort, Merge Sort гарантує складність $O(n \log n)$ у будь-якому випадку — як у найкращому, так і в найгіршому. Основна ідея полягає у рекурсивному розділенні масиву на дві половини до тих пір, поки кожна частина не міститиме лише один елемент, а потім у злитті цих частин у правильному порядку.

Розглянемо роботу алгоритму на прикладі масиву [5, 3, 8, 1]:

1. Масив розділяється на дві частини: [5, 3] і [8, 1]
2. Кожна частина розділяється знову: [5], [3] і [8], [1]
3. Починається злиття — порівнюються елементи і розміщуються у правильному порядку: [3, 5] і [1, 8]
4. Злиття двох відсортованих частин: порівнюємо 3 і 1 $\rightarrow$ 1, потім 3 і 8 $\rightarrow$ 3, потім 5 і 8 $\rightarrow$ 5, залишається 8
5. Результат: [1, 3, 5, 8]

Часова складність Merge Sort стабільно становить $O(n \log n)$ у всіх випадках.

Проте алгоритм має суттєвий недолік — він не є in-place. Для злиття частин потрібна додаткова пам'ять $O(n)$, що може бути критичним при роботі з великими масивами даних.

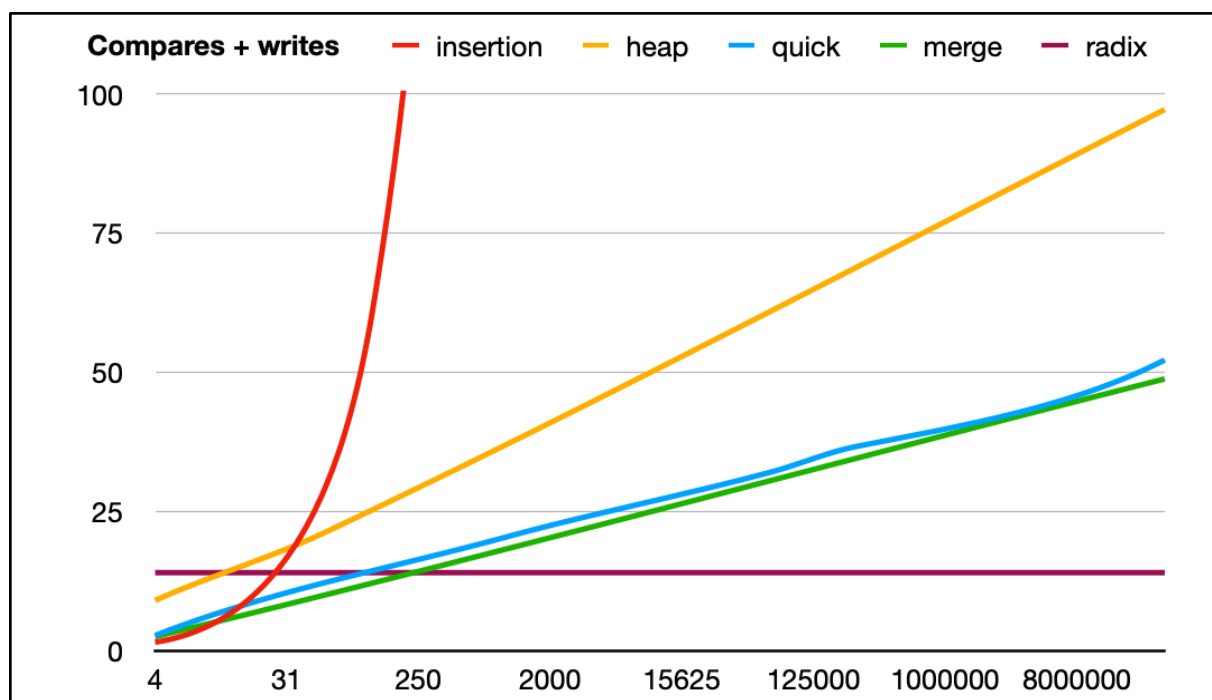


Рисунок 1.3 - Візуалізація порівняння ефективності алгоритмів сортування [13]

На рисунку зображено показники часу сортування відносно кількості елементів. Bubble Sort має аналогічну до Insertion Sort складність $O(n^2)$, тому на графіку поведився б схожим чином. Як ми бачимо, найефективнішим є алгоритм radix, проте це не порівнюваний алгоритм, тому він може використовуватись в обмежених випадках. Далі йдуть Quicksort та Merge Sort, де Merge Sort все ж

трішечки швидше, проте слід пам'ятати, що для сортування даних Merge Sort потребує значний об'єм пам'яті, а якщо це мільйони елементів, то це величезний об'єм додаткових ресурсів. Тому Quick Sort можна вважати найефективнішим алгоритмом сортування серед усіх універсальних. Quicksort є in-place, тобто використовує рівно стільки пам'яті, скільки займають елементи, та доволі швидко сортує будь-який тип даних.

1.3 Аналіз часової та просторової складності

При виборі алгоритму сортування для конкретної задачі недостатньо знати лише принцип його роботи — необхідно розуміти наскільки ефективно він використовує два ключових ресурси: час та пам'ять. Для формального опису ефективності алгоритмів використовують асимптотичний аналіз.

Асимптотичний аналіз дозволяє оцінити як змінюється час виконання або обсяг пам'яті алгоритму залежно від кількості вхідних даних n . Для цього використовують O -нотацію (Big O), яка описує верхню межу зростання — тобто найгірший можливий випадок. Наприклад $O(n^2)$ означає що при збільшенні кількості елементів вдвічі, час виконання зростає приблизно вчетверо.

На практиці розрізняють три сценарії роботи алгоритму:

1. найкращий випадок — вхідні дані вже впорядковані або близькі до впорядкованих;
2. середній випадок — вхідні дані розташовані випадковим чином;
3. найгірший випадок — вхідні дані розташовані так, що алгоритм працює максимально неефективно.

У таблиці 1.1 наведено порівняння часової та просторової складності алгоритмів сортування реалізованих у SortBench.

З таблиці видно кілька важливих закономірностей. Bubble Sort, Selection Sort та Insertion Sort мають складність $O(n^2)$ у середньому випадку — це означає що на великих масивах вони працюють значно повільніше. Проте Insertion Sort та Bubble Sort мають складність $O(n)$ у найкращому випадку, що робить їх ефективними для майже відсортованих даних.

Таблиця 1.2 - Порівняння складності алгоритмів сортування

Алгоритм	Найкращий	Середній	Найгірший	Пам'ять	Стабільний
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Так
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ні
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Так
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Ні
Quick Sort (median-of-three)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	Ні
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Так

Quick Sort та Merge Sort належать до значно ефективніших алгоритмів зі складністю $O(n \log n)$ у середньому випадку. Головна відмінність між ними — використання пам'яті. Merge Sort потребує додаткову пам'ять $O(n)$ для злиття частин, тоді як Quick Sort є in-place алгоритмом і використовує лише $O(\log n)$ пам'яті для стеку рекурсії. Саме тому Quick Sort з оптимізацією median-of-three вважається найбільш практичним універсальним алгоритмом сортування.

Стабільність алгоритму також є важливою характеристикою. Стабільний алгоритм зберігає відносний порядок елементів з однаковими значеннями. Наприклад якщо сортувати список студентів за оцінкою, стабільний алгоритм гарантує що студенти з однаковою оцінкою залишаться у тому самому відносному порядку що й до сортування. [14]

1.4 Огляд існуючих рішень для візуалізації алгоритмів

Для ефективного вивчення алгоритмів сортування використовують візуальне представлення сортування елементів. Для цього розроблено безліч приватних рішень, та багато відкритих безкоштовних інструментів. Такі інструменти дозволяють в реальному часі побачити роботу бажаного алгоритму сортування на прикладі не відсортованого набору даних. Такі інструменти часто дозволяють вибрати алгоритм сортування, змінити набір вхідних даних та швидкість відтворення. Найпопулярнішим вибором таких інструментів є:

- Visualgo
- Toptal Sorting Algorithms Animations
- Algorithm Visualizer
- The Sound of Sorting
- USFCA Data Structure Visualizations

VisuAlgo - це безкоштовний веб-ресурс розроблений у Національному університеті Сінгапуру під керівництвом доктора Стівена Халіма. Платформа надає інтерактивну візуалізацію широкого спектру алгоритмів та структур даних, серед яких алгоритми сортування займають центральне місце. [2]

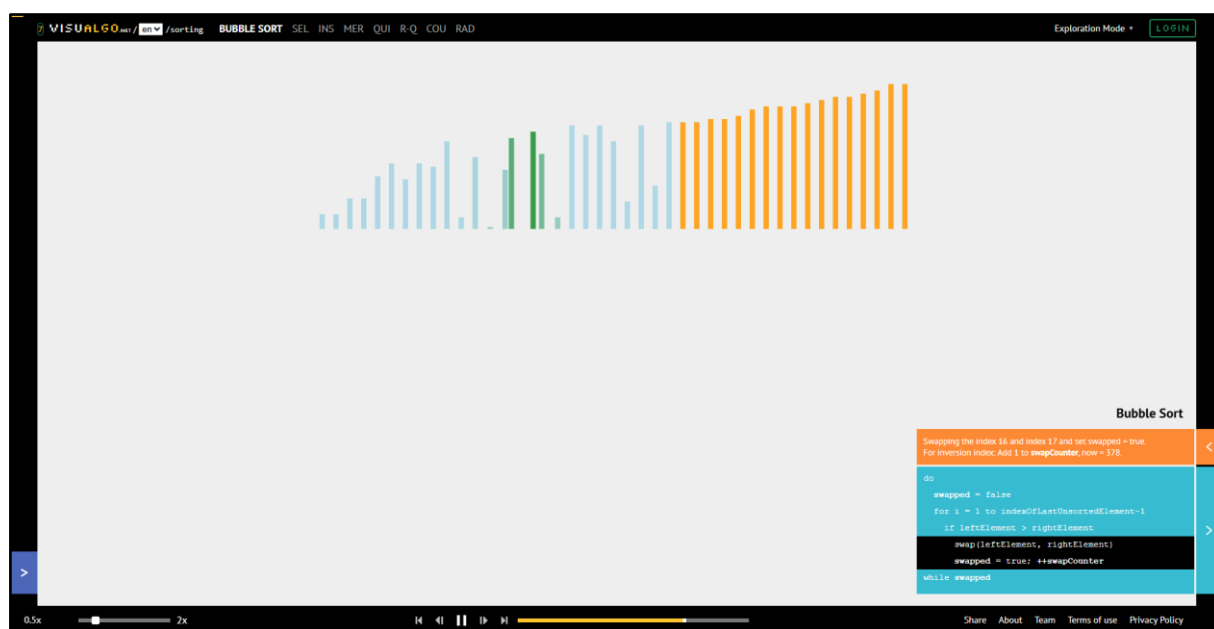


Рисунок 1.4 - Сайт VisuaAlgo

VisuAlgo дозволяє користувачу вибрати алгоритм сортування, налаштувати вхідні дані та швидкість відтворення, а також переглянути покрокове виконання алгоритму з детальними поясненнями кожного кроку. Платформа підтримує широкий набір алгоритмів сортування - Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort та інші.

Головним недоліком VisuAlgo є відсутність можливості додавання власних реалізацій алгоритмів - користувач може лише спостерігати за роботою попередньо визначених алгоритмів. Крім того, платформа є виключно веб-орієнтованим рішенням і потребує постійного підключення до інтернету.

Toptal Sorting Algorithms Animations - це веб-ресурс розроблений платформою Toptal, що спеціалізується на візуальному порівнянні алгоритмів сортування. Головною особливістю ресурсу є можливість одночасного перегляду роботи кількох алгоритмів сортування на одному екрані, що дозволяє наочно порівняти їх швидкість та поведінку на однакових вхідних даних. [3]

Ресурс підтримує різні типи вхідних даних - випадковий масив, майже відсортований, відсортований у зворотному порядку та масив з однаковими елементами. Це дозволяє побачити як поводить себе кожен алгоритм у різних сценаріях.

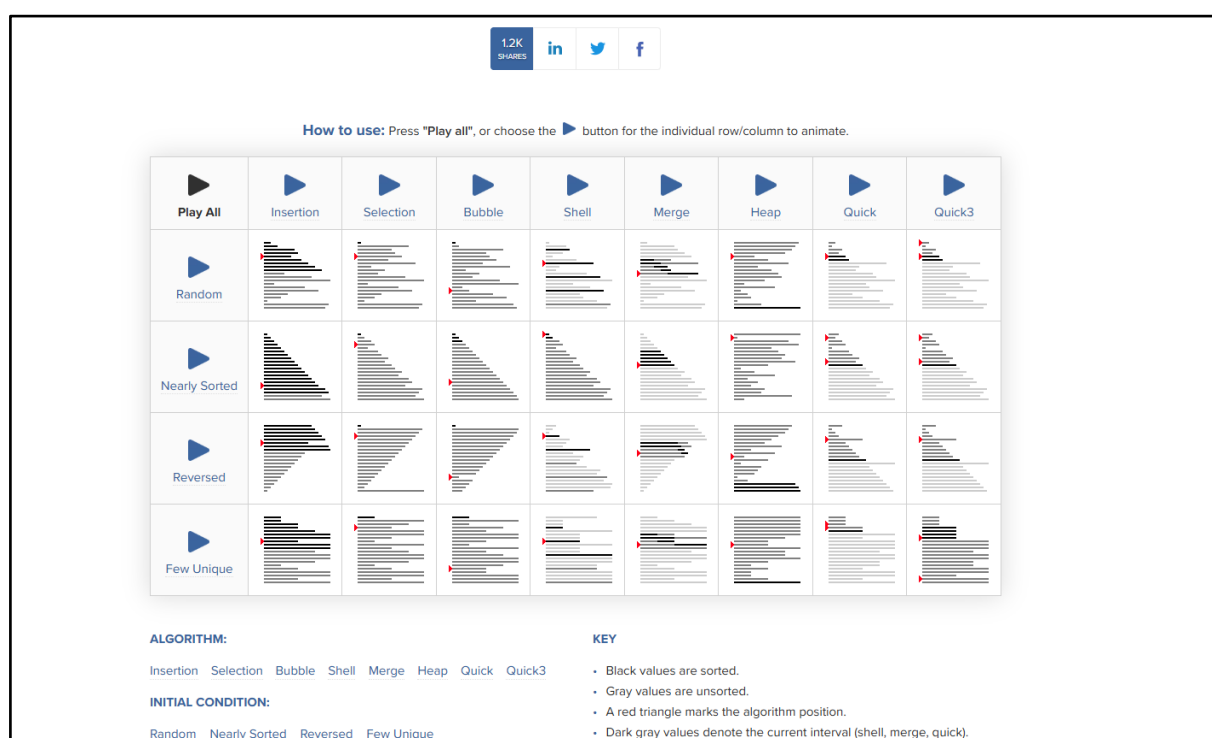


Рисунок 1.5 - Сайт Toptal Sorting Algorithms Animation

Проте Toptal Sorting Algorithms Animations має суттєві обмеження. Ресурс надає лише попередньо визначений набір алгоритмів без можливості додавання власних реалізацій. Відсутня також можливість керування швидкістю відтворення та покрокового перегляду - анімація відтворюється автоматично без можливості паузи на конкретному кроці.

Algorithm Visualizer - це проєкт з відкритим вихідним кодом, що надає інтерактивне середовище для візуалізації різноманітних алгоритмів, серед яких

алгоритми сортування, обходу графів, динамічного програмування та інші. На відміну від попередніх ресурсів, Algorithm Visualizer дозволяє користувачу самостійно реалізувати власний алгоритм та візуалізувати його роботу. Платформа дозволяє використовувати три мови програмування: Java, JavaScript, C++. [4]

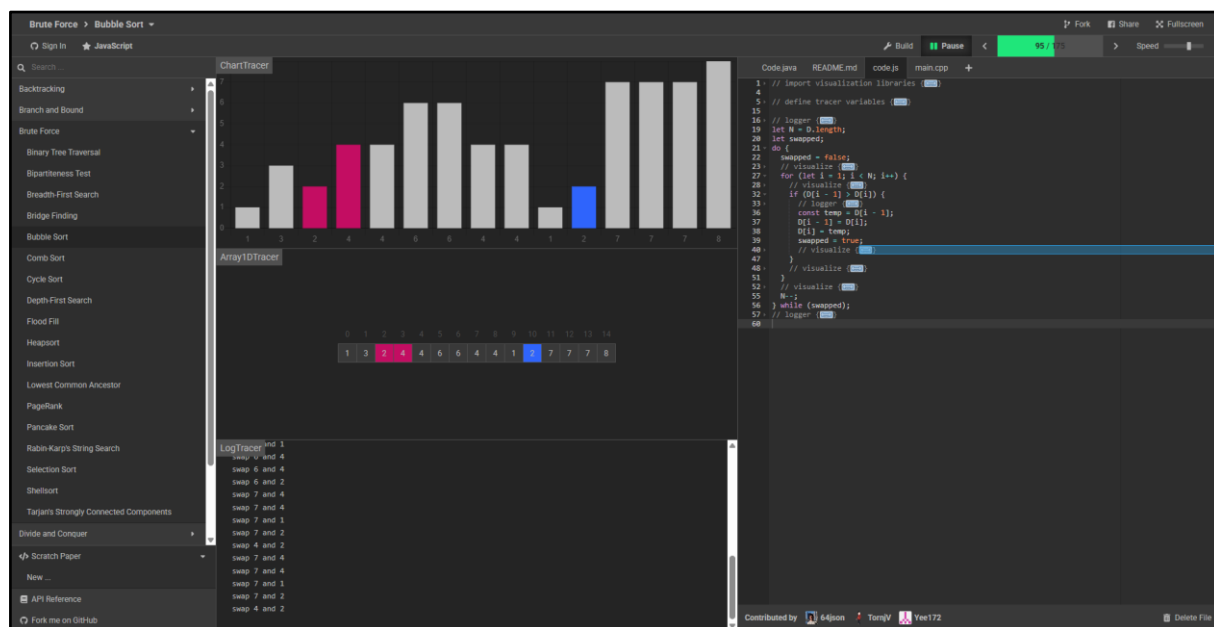


Рисунок 1.6 - Сайт Algorithm Visualizer

Платформа надає користувачу спеціальний API для керування візуалізацією - користувач пише код алгоритму і одночасно визначає як саме має відображатись кожен крок на екрані. Це робить Algorithm Visualizer значно гнучкішим інструментом порівняно з VisuAlgo та Toptal.

Проте така гнучкість має свою ціну. Користувач змушений одночасно думати про логіку алгоритму та про логіку його відображення, що значно збільшує когнітивне навантаження. Це може призводити до хибного розуміння алгоритму, адже увага розподіляється між двома різними задачами.

The Sound of Sorting - це настільний застосунок з відкритим вихідним кодом, розроблений Тімо Бінгманном у 2013 році. Головною особливістю застосунку є те, що він не лише візуалізує роботу алгоритмів сортування, а й озвучує кожну операцію порівняння та обміну елементів - звук змінюється залежно від значення елемента, що створює унікальний аудіовізуальний досвід. [5]

Застосунок підтримує великий набір алгоритмів сортування - понад 20 різних реалізацій, включаючи рідкісні та екзотичні алгоритми яких немає в інших інструментах. Користувач може налаштовувати швидкість відтворення, розмір масиву та тип вхідних даних.

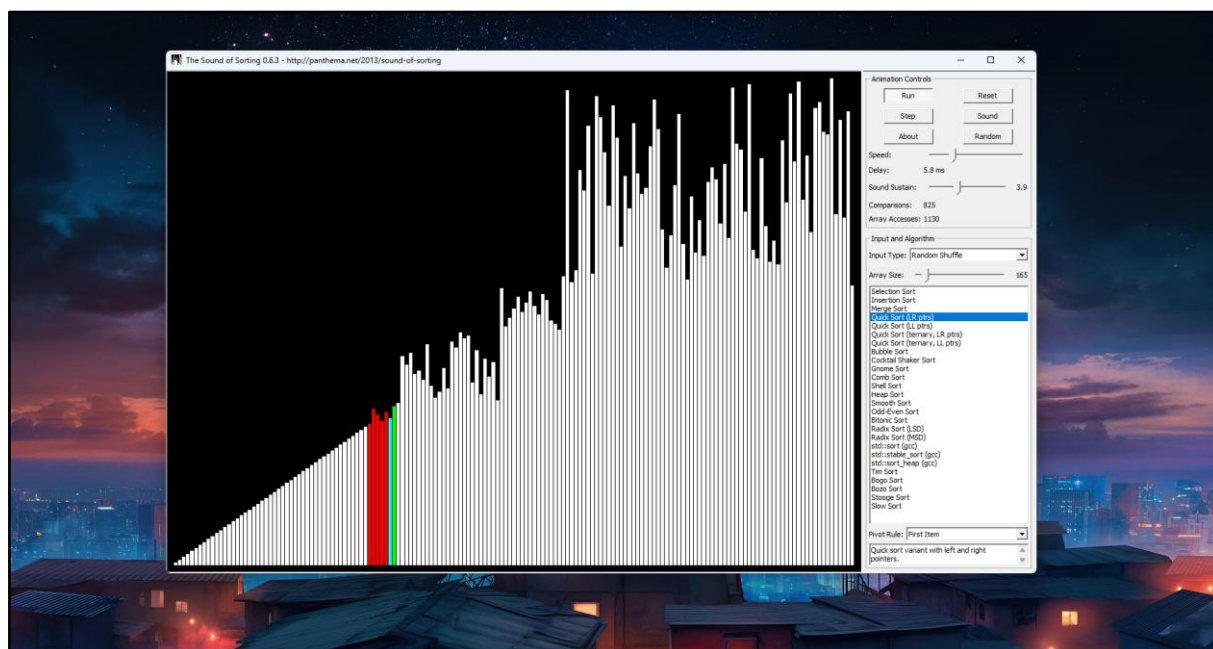


Рисунок 1.7 - Вигляд програми для сортування The Sound of Sorting

Проте The Sound of Sorting також має обмеження - він надає лише попередньо визначений набір алгоритмів без можливості додавання власних реалізацій. Крім того, застосунок не оновлювався з 2013 року, що може спричинити проблеми із запуском на сучасних операційних системах.

USFCA Data Structure Visualizations - це веб-ресурс розроблений у Університеті Сан-Франциско, що надає візуалізацію широкого спектру структур даних та алгоритмів. Ресурс орієнтований насамперед на навчальні цілі та використовується як допоміжний матеріал у курсах з алгоритмів та структур даних. [6]

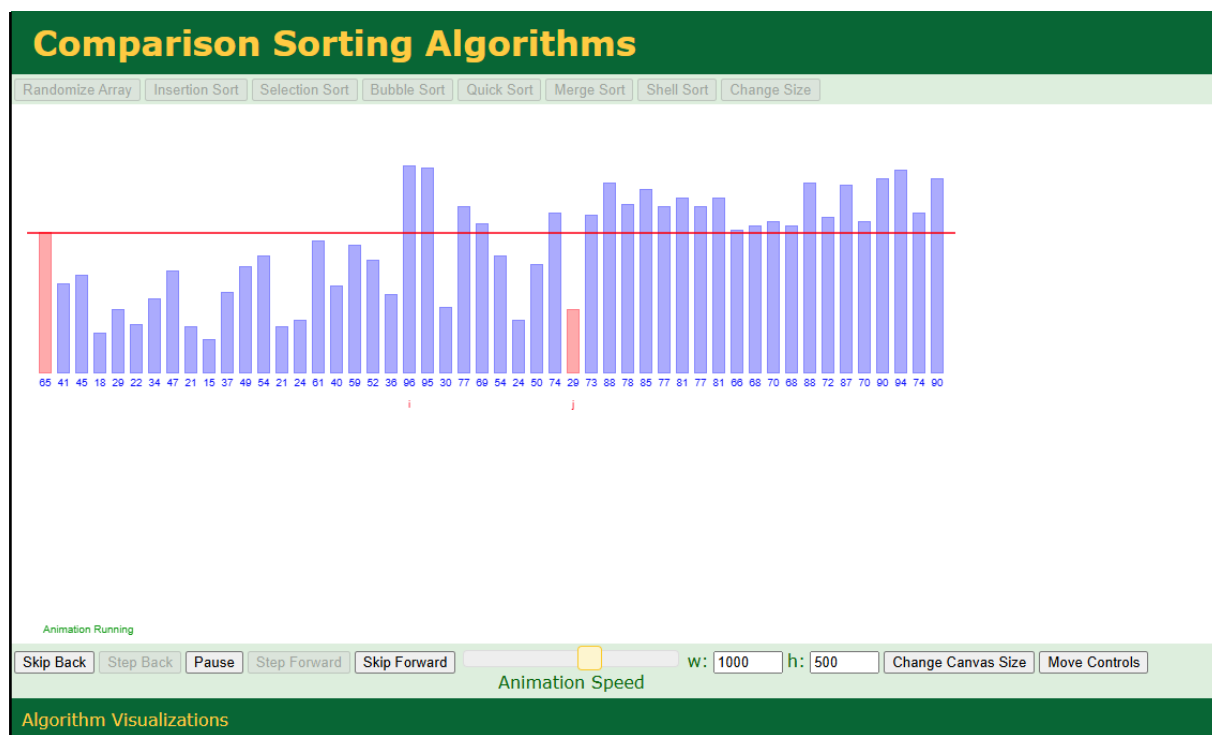


Рисунок 1.8 - Сайт USFCA Data Structure Visualizations

Платформа дозволяє переглянути покрокову роботу алгоритмів сортування з можливістю керування швидкістю відтворення та переходу між кроками. Інтерфейс є мінімалістичним та зрозумілим, що робить ресурс доступним для початківців.

Проте USFCA Data Structure Visualizations має ті самі обмеження що і більшість розглянутих інструментів - користувач може лише спостерігати за роботою попередньо визначених алгоритмів без можливості додавання власних реалізацій.

Таким чином, проведений огляд існуючих рішень для візуалізації алгоритмів сортування виявив спільний недолік більшості з них - відсутність можливості інтеграції власних реалізацій алгоритмів. Єдиним винятком є Algorithm Visualizer, проте його підхід вимагає від користувача одночасної реалізації як логіки алгоритму так і логіки його відображення, що збільшує когнітивне навантаження та відволікає від основної задачі. Виявлені недоліки існуючих рішень обумовлюють доцільність розробки програмної системи, яка забезпечує простий механізм інтеграції власних алгоритмів без необхідності ручного визначення логіки візуалізації. Недоліки існуючих рішень мотивують розробку кращих

рішень, таких як SortBench, що дозволяє розробляти власні алгоритми сортування простим шляхом, думаючи лише про основну логіку алгоритму.

2 ПРОЄКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ SORTBENCH

2.1 Огляд SortBench

Перш ніж ознайомитись з процесом розробки SortBench, варто хоча б зрозуміти як виглядає та що має застосунок. [15]

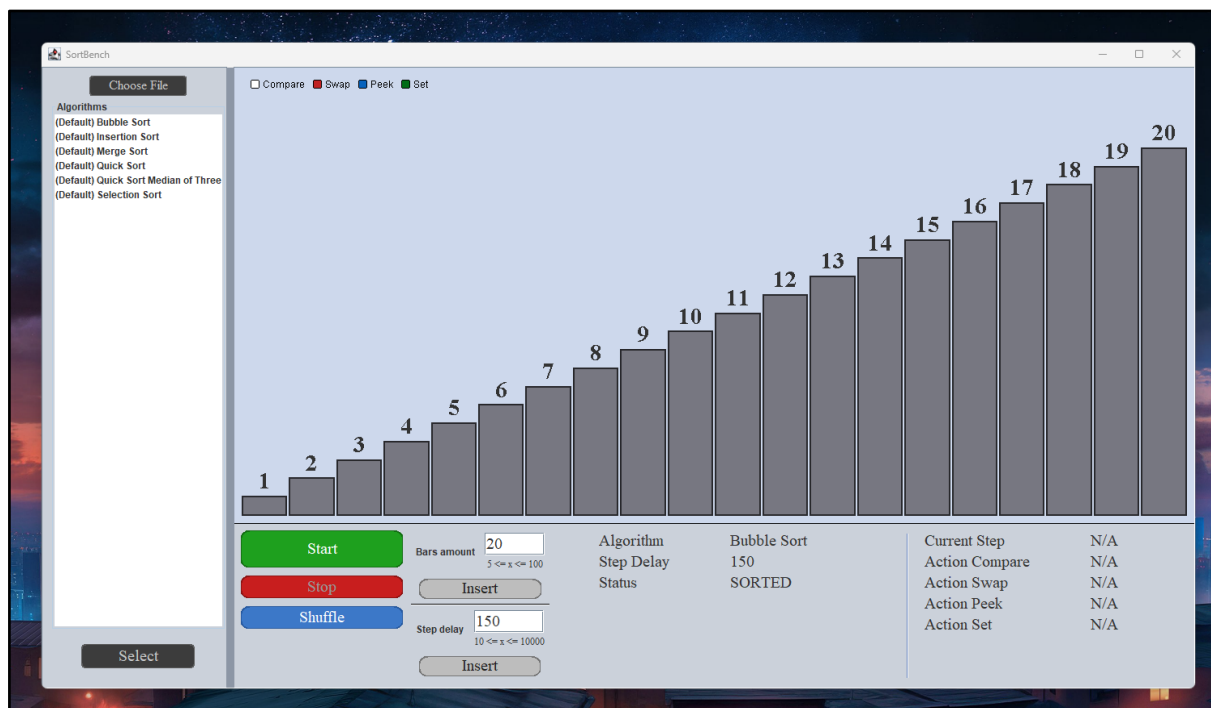


Рисунок 2.1 - SortBench

Як видно на рисунку 2.1, SortBench складається з трьох частин:

1. Зліва - панель вибору алгоритмів сортування;
2. Справа зверху - панель сортування елементів;
3. Справа знизу - панель керування процесом сортування та статистика.

Панель вибору алгоритмів (ліва):

- список вбудованих алгоритмів;
- кнопка для завантаження зовнішнього алгоритму;
- кнопка вибору алгоритму.

Панель сортування (права верхня):

- стовпці що відображають елементи;
- кольорове підсвічування операцій;

- числові значення на стовпцях;
- підказка кольорів.

Панель керування (права нижня):

- кнопки старт/стоп;
- слайдер швидкості;
- слайдер кількості елементів;
- статистика: кроки, compare, swap, peek, set;
- інформаційні маркери стану.

Основною функцією застосунку є візуалізація процесу сортування, де наочно можна побачити виконання дій над стовпцями.



Рисунок 2.2 - Робота SortBench

На рисунку 2.2 зображено роботу застосунку при стандартних налаштуваннях. При стандартних налаштуваннях застосунок виконує сортування елементів з використанням алгоритму сортування Bubble Sort. Кількість елементів при стандартному налаштуванні 20. Стандартна затримка між операціями 150 мілісекунд. Застосунок дозволяє налаштовувати процес та умови сортування з-за допомогою вбудованих модулів на панелі.



Рисунок 2.3 - Робота SortBench з іншими налаштуваннями

На рисунку 2.3 застосунок сортує 100 елементів з використанням Quicksort алгоритму. Сортування відбувається доволі швидко, адже була встановлена майже мінімальна затримка між операціями, а саме 30 мілісекунд.

SortBench пропонує свою ультимативну функцію, а саме можливість додавати власні алгоритми сортування.

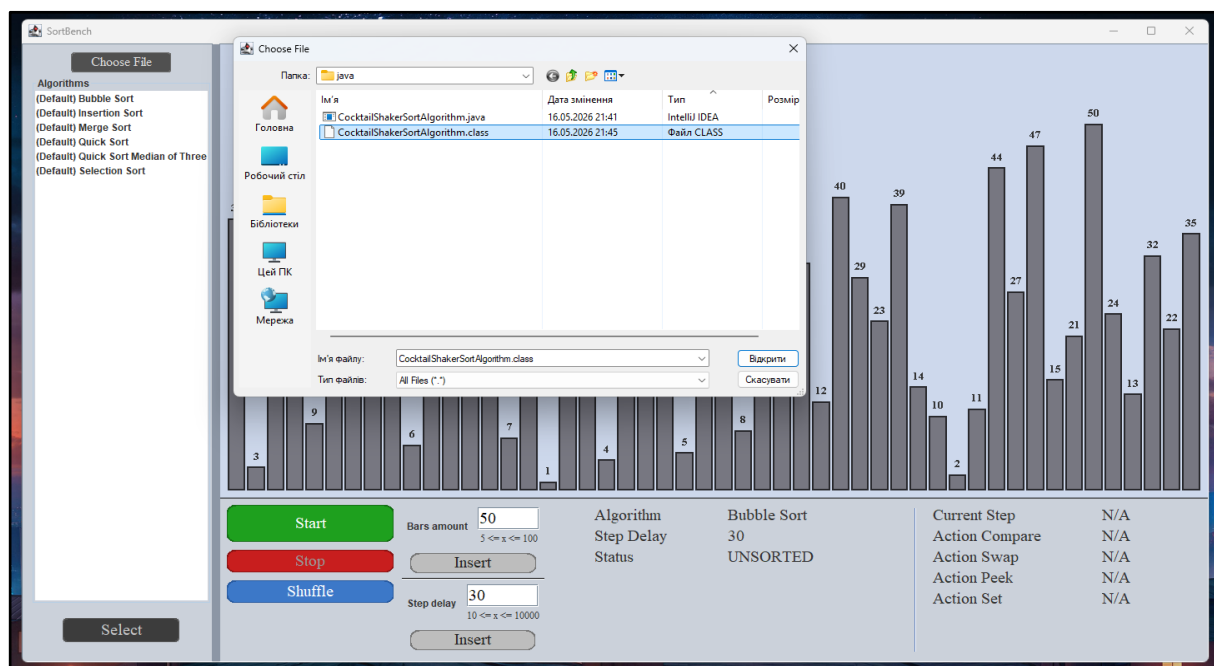


Рисунок 2.4 - Додавання власного алгоритму сортування

На рисунку 2.4 ми бачимо вікно файлової системи, де ми обираємо алгоритм сортування для додавання в застосунок.

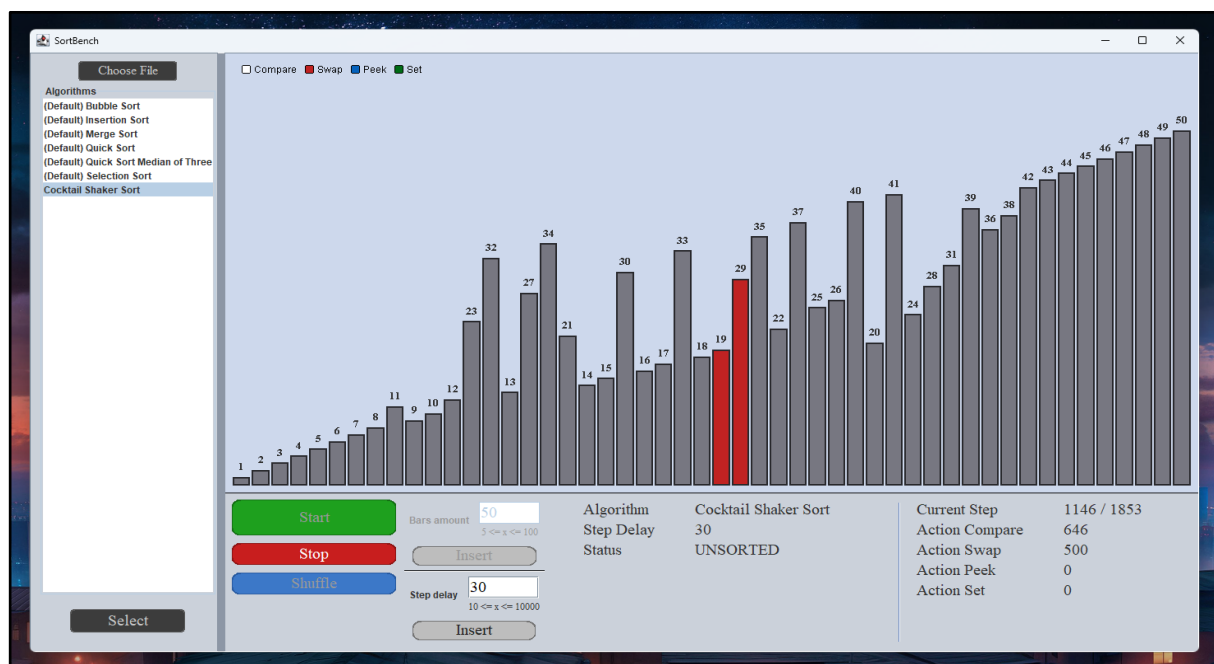


Рисунок 2.5 - Робота зовнішнього алгоритму Cocktail Shaker Sort

На рисунку 2.5 зображено роботу алгоритму Cocktail Shaker Sort, що не був базово представлений в програмі, а був доданий до програми через опцію “Choose File”.

Таким чином, SortBench є готовим інструментом для візуалізації та вивчення алгоритмів сортування, що підтримує як вбудовані алгоритми, так і власні реалізації користувача. Детальний опис процесу розробки та архітектурних рішень наведено в наступних підрозділах.

2.2 Визначення вимог та планування MVP

Будь-який проект що розробляється має спочатку стати MVP (Minimum Viable Product). Це сприяє стабільній та контрольованій розробці. MVP дозволяє отримати мінімальний результат за мінімальні зусилля, де такий проект стає каркасом для подальшого розвитку.

Щоб розуміти, що маєтись на увазі MVP, треба було визначити мінімальні вимоги та основні функції проєкту. Це дозволяє чітко розуміти як має виглядати застосунок та почати планування щодо його реалізації.

На цьому етапі треба сконцентруватись на пошуку конкретних речей, де чим більше знайдеш конкретики, тим простіше буде розробляти продукт. На момент формування мінімальних вимог SortBench було визначено наступні пункти:

1. Настільний застосунок;
 - a. можна змінювати розмір вікна
 - b. має мінімальний розмір
 - c. панелі в застосунку є динамічними
2. Має дисплей з відображенням сортування елементів;
 - a. елементи у вигляді стовпців
 - b. стовпці мають числове відображення елементу
 - c. стовпці змінюють колір при операціях над ними
3. Має дисплей для налаштування процесу сортування;
 - a. можна змінювати швидкість відтворення сортування
 - b. можна змінювати кількість стовпців
 - c. можна запускати сортування
 - d. можна зупиняти сортування
 - e. можна тасувати елементи
4. Можна обирати алгоритм для сортування;
 - a. є вбудовані алгоритми для тестового сортування
 - b. розробка окремої абстракції для розробки зовнішніх алгоритмів сортування та публікація на Maven Central Repository
 - c. є кнопка для додавання зовнішнього алгоритму сортування

Також було зрозуміло, що створення користувацького інтерфейсу буде відбуватись з-за допомогою UI бібліотеки Swing. Swing має багато готових інструментів, що дозволяє легко створити потрібний UI для програми використовуючи Java. [16]

Виконання цих вимог дозволяє отримати мінімально готовий застосунок, функціонал якого можна розширювати. Наприклад, ось такі функції були надбудовані після отримання MVP:

1. Статистика

- a. Кількість пройдених кроків
 - b. Кількість операцій Compare
 - c. Кількість операцій Swap
 - d. Кількість операцій Peek
 - e. Кількість операцій Set
2. Зворотній зв'язок
- a. Інформаційний маркер обраного алгоритму
 - b. Інформаційний маркер швидкості відтворення сортування
 - c. Інформаційний маркер стану сортування
3. Підказки
- a. Підказка відповідності кольорів до операцій над елементами

Додаткові функції не були передбачені на початку проектування, щоб сконцентрувати зусилля на головному. До, під час, та після розробки MVP з'являлось багато ідей, але вони могли бути реалізовані тільки після доведення проекту до стану MVP. Це дозволило розробити основу застосунку та зберігати мотивацію впродовж розробки.

Також, під час усієї розробки, прямо в проекті вівся допоміжний текстовий документ, що допомагав зрозуміти поточні та виконані задачі. Цей список вівся в довільному порядку, але головну роль він виконував на відмінно - розбивати завдання на підзавдання. Оригінальна файл досі залишається частиною проекту, тож будь-хто може знайти його на GitHub за посиланням <https://github.com/BohdanYarovy/Sort-Bench-Application/blob/main/notes.txt>.

```

1  [🔗] Create a window
2  [+] Create window frame
3  [+] Make window and panels resizable
4  [+] Implement visualization display
5      [+] Resizable actionResponsives
6      [+] Resizable actionResponsive labels with value
7  [+] Implement MVP statistic display
8      [+] Place button 'Start'
9      [+] Place button 'Stop'
10 [+] Integrate multi-algorithmic choice
11 [+] Write 'Bubble sort'
12 [+] Force it to sort actionResponsives on click buttons
13 [+] Create structure of data for manipulate list of data and change actionResponsive state
14      [+] Find out a structure of data that can work with this system
15 [+] Clean up the project
16 [+] Refactor panel management
17     [+] Refactor AlgorithmCollectionPanel
18     [+] Refactor AlgorithmDashboardPanel
19     [+] Refactor SortingDisplayPanel
20     [+] Refactor StatisticMonitorPanel
21     [+] Reorganize them using GridBagLayoutManager instead of custom AdaptivePanel abstract class
22 [+] Refactor BarAlignmentController and SortingDisplayDrawer to show actionResponsives more accuracy
23 [+] Rework SimpleSortingController and add more methods on it
24 [+] Add changing amount of components
25 [+] Add step counter
26 [+] Add speed pane
27 [+] Refactor code

```

Рисунок 2.6 - Перша частина файлу notes.txt

```

28 [+] Place all components on the pane
29     [+] Add some info on the Control panel about current state of the program
30         On the left side we should have buttons and other control components. On the right side
31         we should have components that show information about current state of the program or
32         algorithm execution process (what current delay, what status {sorted, unsorted}, steps,
33         what algorithm is using, ).
34 [+] Add hints on sorting screen
35 [+] Implement algorithm collection pane
36     [+] Add scrollable list
37     [+] Add default algorithms
38     [+] Add algorithm chose button
39     [+] Allow choose algorithm in runtime
40     [+] Allow user to add custom .class file with implementation of SortAlgorithm
41     [+] Add button for choosing new algorithm from file directory
42     [+] Create external API for using it for creating external .class files
43     [+] Create a few algorithms with external API
44     [+] Refactor code
45 [+] Create LIB for adding any sorting algorithm to program
46 [+] Beautify the application
47 [+] Publish external LIB on the Maven Central
48     [+] Write Javadoc in LIB classes
49     [+] Organize pom file
50     [+] Publish
51 [+] Final code refactoring
52 [🔗] Write README file
53 [-] ... (add new ones after reviewing the project)

```

Рисунок 2.7 - Друга частина файлу notes.txt

2.3 Архітектура системи

SortBench було розроблено з використанням класичного патерну проєктування MVC, де система чітко розподіляє ролі між компонентами. Верхній рівень системи представлений головним фреймом, на якому розташовані дві основні панелі.

Перша панель - панель вибору алгоритму сортування. Вона містить список доступних алгоритмів та елементи керування для їх вибору і завантаження. Друга панель - панель сортування, яка складається з двох підпанелей: панелі відображення сортування та панелі налаштування процесу сортування.

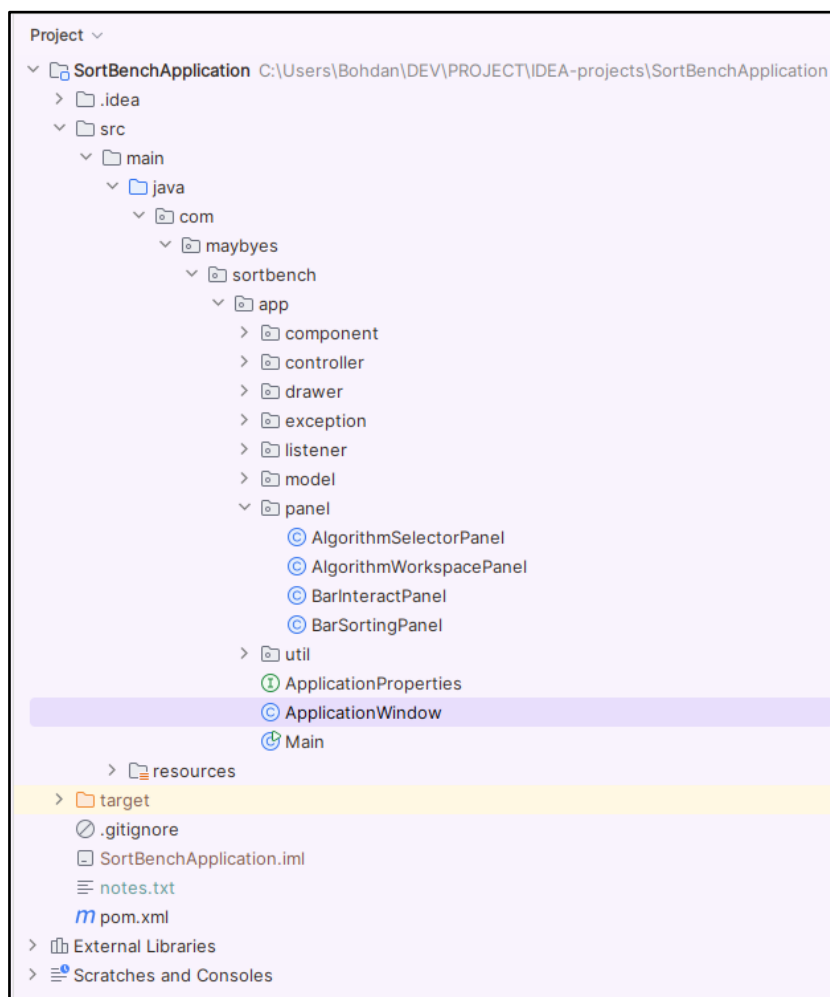


Рисунок 2.8 - Файлове відображення проекту

Головний фрейм в проекті має ім'я `ApplicationWindow`, і його роль полягає в тому, щоб створити вікно з правильними налаштуваннями, створити панелі `AlgorithmSelectorPanel` та `AlgorithmWorkspacePanel` та правильно їх розташувати на фреймі. Фрейм створюється з розмірами відносно розмірів екрану. Для розташування панелей, використовується `JSplitPane`, що дозволяє користувачу змінювати розмір панелей в фреймі під час Runtime. Усі налаштування фрейму винесені в константи в класі `ApplicationWindow` або в спеціальний клас для всіх констант `ApplicationProperties`.

AlgorithmSelectorPanel являє собою панель для вибору алгоритмів сортування. Під час запуску застосунку, в цю панель має завантажитись вбудовані алгоритми сортування. На панелі є кнопки, що дозволяють відкрити файлову систему, де користувач може обрати скомпільований клас, який є нащадком класу SortAlgorithm. Ця панель має спеціальний JList<T>, що дозволяє представити алгоритми у вигляді списку для вибору. Через те що ця панель знаходиться в JSplitPane, то цю панель можна робити ширше чи вужче за рахунок розміру правої панелі.

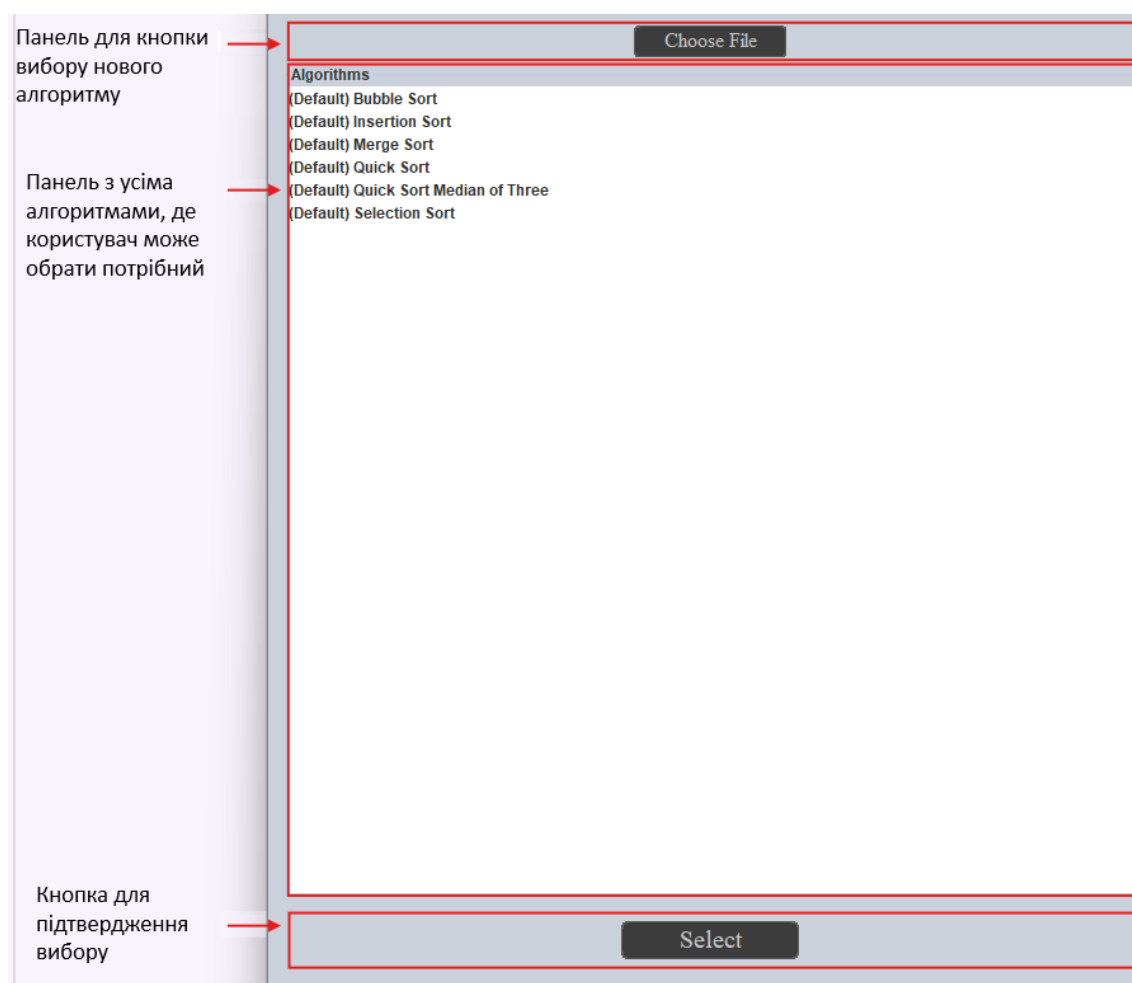


Рисунок 2.9 - AlgorithmSelectionPanel

Вибір алгоритму відбувається за рахунок вибору елементу зі списку алгоритмів, та підтвердження натиском на кнопку Select. AlgorithmSelectorPanel має спеціальний слухач, який додається до кнопки Select, де при натиску на кнопку, цей слухач активується. Слухач є nullable полем

об'єкту `AlgorithmSelectorPanel`, і має бути встановлений сеттером зовні після ініціалізації класу. `ApplicationWindow` встановлює слухач після ініціалізації, а саме передає праву панель `AlgorithmWorkspacePanel`, що реалізує інтерфейс `SelectSortAlgorithmListener`. Таким чином відбувається вибір правильного алгоритму під час роботи програми. Якщо не встановити слухач вибору алгоритму, то можливості обрати алгоритм сортування не буде.

`AlgorithmWorkspacePanel` є правою панеллю, що надає доступ до налаштування роботи сортування та безпосередньо демонструє сортування. Відповідно ця панель містить в собі дві інші панелі:

1. `BarSortingPanel` - панель для відображення сортування;
2. `BarInteractPanel` - панель для відображення статистики та налаштування процесу сортування.

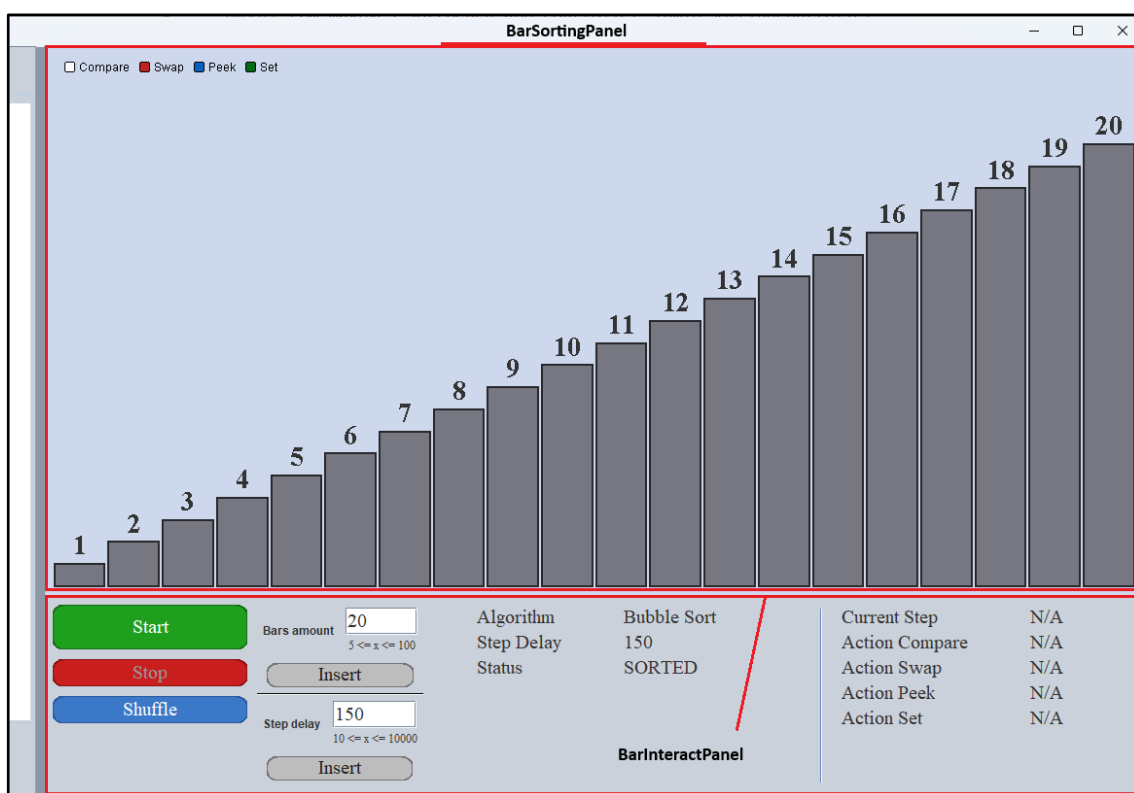


Рисунок 2.10 - `AlgorithmWorkspacePanel`

SortingController по факту є головним зв'язуючим вузлом, без якого нічого працювати просто не буде. Цей контроллер відповідає за усі операції сортування, та має ось такий вигляд:

Лістинг 2.1 - Клас SortingController

```
public interface SortingController {  
  
    void start();  
  
    void stop();  
  
    void shuffle();  
  
    void setBarsAmount(int amount);  
  
    void setDelay(int delay);  
  
}
```

Як видно з лістингу 2.1, контроллер має функції для керування сортуванням, а також функції для встановлення значення кількості стовпців та значення затримки.

SimpleSortingController реалізує всі функції SortingController, де зберігає всю інформацію про сортування в SortingContext, всі статистичні дані записуються з допомогою MetricManager, а панель відображення сортування оновлюється через UpdateBarsOnScreenListener. SortingContext об'єкт зберігає в собі інформацію про стовпці, поточний алгоритм сортування та швидкість відтворення сортування. Функція start() запускає процес сортування, де використовується вищезгаданий контекст сортування для оновлення даних про поточні елементи, алгоритм сортування використовується для сортування елементів, а MetricManager використовується для фіксування статистичних даних. Затримка відбувається з-за допомогою Timer, який виконує всі ці дії один раз в визначений проміжок часу(delay). Кожну ітерацію таймер оновлює панель відображення,

використовуючи для цього `UpdateBarsOnScreenListener`. В розділі 2.4 буде більш глибоке пояснення про механізм сортування та як це відображається покроково.

`BarSortingPanel` - це панель для відображення стовпців. Роль цієї панелі досить проста, а саме прийняти колекцію елементів та намалювати їх в себе на полотні. Важливо розуміти, що кожен елемент знає, де він має розміщуватись, і який розмір він має. Елементу в цьому допомагає `BarAlignmentController`, який рахує позицію та розмір для стовпця, відносно кількості елементів та розміру полотна. Коли кожен елемент знає який розмір має та на яких координатах має розташовуватись, то не проблема його відобразити. Для цього використовується `SortingDisplayBarDrawer`, компонент що розуміється на відображенні компонентів.

`BarSortingPanel` також містить підказки для користувача, який колір відповідає якій операції над елементом. Такі підказки малює для цієї панелі `SortingDisplayHintDrawer`.

`BarInteractPanel` містить в собі 3 панелі керування та аудиту. Ці панелі допомагають правильно розмістити панелі, та логічно інкапсулювати їх у коді.

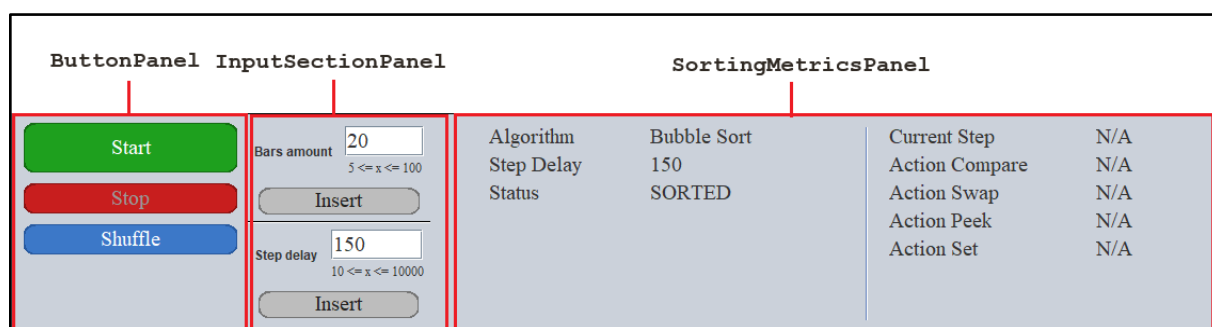


Рисунок 2.11 - `BarInteractPanel`

Всі кнопки, що розміщені в `ButtonPanel` напряду викликають метод `SortingController`. Введені значення в поля `InputSectionPanel` так само використовують `SortingController` для того, щоб встановити нові параметри сортування. `SortingMetricsPanel` періодично оновлює відображену

статистику, беручи дані з постачальників, який надає SimpleSortingController.

Як було вище зазначено, що кожен елемент знає, де він має малюватись, і які розміри він має. Більше того, такий компонент знає в який колір він має малюватись, адже колір змінюється в залежності від виконаної операції над елементом. В системі такий компонент представляють об'єкти класу Bar.

Лістинг 2.2 - Клас Bar

```
@Getter
@Setter
@ToString
public class Bar implements Comparable<Bar> {
    private final int value;

    private Dimension2D barSize;

    private Point2D barPosition;

    private Color innerColor;

    private Color borderColor;

    public Bar(int value) {
        this.value = value;
        this.innerColor = ApplicationProperties.BAR_IDLE_COLOR;
        this.borderColor =
ApplicationProperties.BAR_BORDER_COLOR;
    }

    @Override
    public int compareTo(Bar other) {
        return this.value - other.value;
    }

    public void setCompared() {
        innerColor = ApplicationProperties.BAR_COMPARE_COLOR;
```



```

    }

    public void setSwapped() {
        innerColor = ApplicationProperties.BAR_SWAP_COLOR;
    }

    public void setPeeked() {
        innerColor = ApplicationProperties.BAR_PEEK_COLOR;
    }

    public void setSet() {
        innerColor = ApplicationProperties.BAR_SET_COLOR;
    }

    public void setIdle() {
        innerColor = ApplicationProperties.BAR_IDLE_COLOR;
    }
}

```

Кольори для відображення різних станів в залежності від виконаної операції визначає `ApplicationProperties`.

Ключовим вузлом застосунку є клас `SortingCollection`, що по суті і дозволяє застосунку сортувати елементи. `SortingCollection` це спеціальна абстракція, яка використовується в алгоритмах сортування для маніпуляції над елементами. В кожному реалізованому алгоритмі дії відбуваються виключно через цю колекцію, де кожна виконана дія записується в чергу дій. Таку чергу потім `SimpleSortingController` використовує для того, щоб покроково відтворити сортування над колекцією елементів.

Лістинг 2.3 - Клас `SortingCollection`

```

public interface SortingCollection {

    int compare(int index1, int index2);
}

```

```

Integer peek(int index);

void swap(int index1, int index2);

void set(int index, int value);

int getSize();

}

```

Події виконані над елементами відображаються у вигляді абстракції `Action`, де кожен подію потім можна відтворити викликом методу `perform(List<Bar> elements)`.

Лістинг 2.4 - Клас `Action`

```

public interface Action {

void perform(List<Bar> elements);

}

```

2.4 Розробка застосунку

Графічний застосунок `SortBench` побудований навколо кількох ключових компонентів, що забезпечують весь процес візуалізації сортування: `SortingCollection`, `SimpleSortingController`, `BarAlignmentController` та механізм завантаження алгоритмів. Розуміння роботи цих компонентів дає повне уявлення про те, яким чином застосунок перетворює абстрактний алгоритм сортування на покрокову візуалізацію на екрані. По суті, ці ключові компоненти і були основним викликом під час розробки `SortBench`.

Механізм `record-and-replay`. `SortingCollection`

Центральною ідеєю візуалізації в `SortBench` є підхід `record-and-replay`. Суть підходу полягає в тому, що алгоритм сортування не маніпулює безпосередньо над

графічними елементами - він працює з абстракцією `SortingCollection`, яка записує кожну виконану дію в чергу.

`SortingCollectionImpl` реалізує інтерфейс `SortingCollection` і зберігає два внутрішніх поля: список цілочисельних значень `elements` та чергу дій `actions`. Кожен виклик методу колекції - `compare()`, `swap()`, `peek()`, `set()` - виконує реальну операцію над списком значень і одночасно додає відповідний об'єкт дії в чергу. Наприклад, виклик `swap(i, j)` фізично міняє місцями елементи у списку та додає `SwapAction(i, j)` в чергу.

Лістинг 2.5 - Клас `SortingCollectionImpl`

```
public class SortingCollectionImpl implements SortingCollection
{
    private final List<Integer> elements;

    private final Queue<Action> actions;

    public SortingCollectionImpl(List<Integer> initialElements)
    {
        this.elements = new ArrayList<>(initialElements);
        this.actions = new LinkedList<>();
    }

    @Override
    public int compare(int index1, int index2) {
        Integer element1 = elements.get(index1);
        Integer element2 = elements.get(index2);
        actions.add(new CompareAction(index1, index2));

        return element1 - element2;
    }

    @Override
    public Integer peek(int index) {
        actions.add(new PeekAction(index));
    }
}
```

```

        return elements.get(index);
    }

    @Override
    public void swap(int index1, int index2) {
        Integer temp = elements.get(index1);
        elements.set(index1, elements.get(index2));
        elements.set(index2, temp);

        actions.add(new SwapAction(index1, index2));
    }

    @Override
    public void set(int index, int value) {
        elements.set(index, value);

        actions.add(new SetAction(index, value));
    }

    @Override
    public int getSize() {
        return elements.size();
    }

    public Queue<Action> getActions() {
        return new LinkedList<>(actions);
    }
}

```

Таким чином, після завершення роботи алгоритму список `elements` є відсортованим, а черга `actions` містить повний запис усіх виконаних операцій у правильному порядку. Цей запис і є основою для подальшого покрокового відтворення на екрані. Кожна дія реалізує інтерфейс `Action` з єдиним методом `perform(List<Bar> elements)`, що застосовує відповідну операцію до списку графічних елементів та змінює їх колір відповідно до типу операції.

ЛІСТИНГ 2.6 - Клас CompareAction

```
public record CompareAction(  
    int index1,  
    int index2  
) implements Action {  
  
    @Override  
    public void perform(List<Bar> elements) {  
        Bar bar1 = elements.get(index1);  
        Bar bar2 = elements.get(index2);  
  
        bar1.setCompared();  
        bar2.setCompared();  
    }  
  
}
```

ЛІСТИНГ 2.7 - Клас PeekAction

```
public record PeekAction(  
    Integer index  
) implements Action {  
  
    @Override  
    public void perform(List<Bar> elements) {  
        elements.get(index).setPeeked();  
    }  
  
}
```

ЛІСТИНГ 2.8 - Клас SetAction

```
public record SetAction(  
    Integer index,  
    Integer value  
) implements Action {  
  
    @Override  
    public void perform(List<Bar> elements) {
```

```

        elements.set(index, new Bar(value));
        elements.get(index).setSet();
    }
}

```

Лістинг 2.9 - Клас SwapAction

```

public record SwapAction(
    int index1,
    int index2
) implements Action {

    @Override
    public void perform(List<Bar> elements) {
        Bar bar1 = elements.get(index1);
        Bar bar2 = elements.get(index2);

        bar1.setSwapped();
        bar2.setSwapped();

        elements.set(index1, bar2);
        elements.set(index2, bar1);
    }
}

```

Керування процесом сортування. SimpleSortingController

SimpleSortingController є головним контролером застосунку, що координує увесь процес сортування. При виклику методу start() контролер отримує поточний алгоритм, список елементів та затримку з SortingContext, після чого запускає приватний метод sort().

Лістинг 2.10 - Клас SimpleSortingController

```

@Slf4j
public class SimpleSortingController implements
    SortingController {

```

```

private final SortingContext sortingContext;

private final MetricManager metricManager;

private Timer timer;

private UpdateBarsOnScreenListener
updateBarsOnScreenListener;

public SimpleSortingController() {
    this.metricManager = new MetricManager();
    this.sortingContext = new SortingContext();
}

@Override
public void start() {
    log.debug("Sorting process was started");
    SortAlgorithm algorithm = sortingContext.getAlgorithm();
    int timerDelay = sortingContext.getTimerDelay();
    List<Bar> bars = sortingContext.getBars();
    Queue<Action> actions = sort(algorithm);
    metricManager.reset(actions.size());

    timer = new Timer(timerDelay, e -> {
        if (actions.isEmpty()) {
            timer.stop();
            sortingContext.setSorted(true);

            BarUtil.setIdleBars(bars);
            updateBarsOnScreenListener.update(bars);
            log.info("Bars sorting process finished");
            return;
        }
        BarUtil.setIdleBars(bars);
        Action action = actions.remove();
        action.perform(bars);
        metricManager.recordStep(action);
    });
}

```

```

        updateBarsOnScreenListener.update(bars);
    });
    timer.start();
}

private Queue<Action> sort(SortAlgorithm sortAlgorithm) {
    List<Integer> values = sortingContext.getBarsValues();
    SortingCollectionImpl sortingCollection = new
SortingCollectionImpl(values);
    sortAlgorithm.sort(sortingCollection);

    log.debug("Bars were sorted using '{}' algorithm",
sortAlgorithm.getName());
    return sortingCollection.getActions();
}

@Override
public void shuffle() {
    List<Bar> bars = sortingContext.getBars();
    BarUtil.setIdleBars(bars);
    Collections.shuffle(bars);
    updateBarsOnScreenListener.update(bars);
    sortingContext.setSorted(false);

    log.debug("Was shuffled all bars");
}

@Override
public void stop() {
    if (timer != null) {
        timer.stop();

        log.debug("Sorting process was stopped");
    }
}

@Override
public void setBarsAmount(int amount) {

```



```

        sortingContext.initBars(amount);

updateBarsOnScreenListener.update(sortingContext.getBars());

        log.debug("Was set new bars amount to {}", amount);
    }

    @Override
    public void setDelay(int delay) {
        sortingContext.setTimerDelay(delay);

        if (timer != null) {
            timer.setDelay(delay);

            log.debug("Timer delay was set with new value {}",
delay);
        }
    }

    public void setAlgorithm(SortAlgorithm algorithm) {
        if (timer != null) {
            timer.stop();
        }

        sortingContext.setAlgorithm(algorithm);
        log.debug("Was set sorting algorithm '{}'",
algorithm.getName());
    }

    public void
setUpdateBarsOnScreenListener(UpdateBarsOnScreenListener
updateBarsOnScreenListener) {
        this.updateBarsOnScreenListener =
updateBarsOnScreenListener;
    }

    public MetricSupplier getMetricSupplier() {
        return metricManager;
    }

```

```

    }

    public ContextSupplier getContextSupplier() {
        return sortingContext;
    }
}

```

Метод `sort()` створює новий екземпляр `SortingCollectionImpl` на основі поточних значень елементів і передає його в алгоритм сортування. Алгоритм виконує сортування, записуючи всі дії в чергу колекції. Після завершення роботи алгоритму метод повертає заповнену чергу дій.

Далі `start()` створює `javax.swing.Timer` із заданою затримкою. На кожній ітерації таймер виконує такі дії: скидає кольори усіх елементів до початкового стану, витягує наступну дію з черги, виконує її над списком графічних елементів `Bar`, фіксує статистичні дані через `MetricManager` та оновлює панель відображення через `UpdateBarsOnScreenListener`. Коли черга дій стає порожньою, таймер зупиняється і сортування вважається завершеним.

Такий підхід повністю відокремлює логіку алгоритму від логіки відображення - алгоритм нічого не знає про графічний інтерфейс і працює виключно з числами.

Позиціонування елементів. `BarAlignmentController`

`BarAlignmentController` відповідає за розрахунок позиції та розміру кожного стовпця відносно поточного розміру полотна. Це дозволяє коректно відображати елементи незалежно від розміру вікна застосунку. `BarAlignmentController` враховує відступи при розрахунках позиціонуванні. Такі відступи визначені прямо у класі, у вигляді константних значень.

Лістинг 2.11 - Клас `BarAlignmentController`

```

public class BarAlignmentController {
    private static final double BAR_MARGIN_TOP = 100.0;

```

```
private static final double BAR_MARGIN_BOTTOM = 10.0;

private static final double BAR_MARGIN_SIDE = 10.0;

private static final double BAR_HORIZONTAL_GAP = 4.0;

public void align(List<Bar> bars, Dimension2D
environmentSize) {
    double barWidth = calculateBarWidth(bars,
environmentSize);
    double barHeightUnit = calculateHeightUnit(bars,
environmentSize);

    for (int i = 0; i < bars.size(); i++) {
        Bar bar = bars.get(i);
        placeBar(bar, i, barWidth, barHeightUnit,
environmentSize);
    }
}

private double calculateBarWidth(List<Bar> bars, Dimension2D
environmentSize) {
    double spacing = (bars.size() - 1) * BAR_HORIZONTAL_GAP
+ BAR_MARGIN_SIDE * 2;
    double allowedSpace = environmentSize.getWidth() -
spacing;

    return allowedSpace / bars.size();
}

private double calculateHeightUnit(List<Bar> bars,
Dimension2D environmentSize) {
    double allowedSpace = environmentSize.getHeight() -
BAR_MARGIN_TOP - BAR_MARGIN_BOTTOM;
    Bar highestBar = bars.stream()
        .max(Bar::compareTo)
        .orElseThrow(RuntimeException::new);
```

```

        return allowedSpace / highestBar.getValue();
    }

    private void placeBar(Bar bar, int index, double barWidth,
double heightUnit, Dimension2D environmentSize) {
        Dimension2D size = new DoubleDimension(barWidth,
heightUnit * bar.getValue());
        Point2D.Double position = computeBarPlacement(index,
size, environmentSize);

        bar.setBarSize(size);
        bar.setBarPosition(position);
    }

    private Point2D.Double computeBarPlacement(int index,
Dimension2D barSize, Dimension2D environmentSize) {
        double x = index * (barSize.getWidth() +
BAR_HORIZONTAL_GAP) + BAR_MARGIN_SIDE;
        double y = environmentSize.getHeight() -
barSize.getHeight() - BAR_MARGIN_BOTTOM;

        return new Point2D.Double(x, y);
    }
}

```

Метод `align()` приймає список елементів `Bar` та поточний розмір полотна `environmentSize`. Ширина стовпця визначається рівномірним розподілом доступного простору між усіма елементами з урахуванням відступів та горизонтальних проміжків. Висота стовпця розраховується пропорційно до значення елемента - для цього визначається одиниця висоти як відношення доступної висоти полотна до максимального значення серед усіх елементів.

Позиція кожного стовпця розраховується таким чином, що стовпці вирівнюються по нижньому краю полотна, що і створює характерний візуальний ефект гістограми при сортуванні.

Завантаження алгоритмів. BuiltinAlgorithmLoader та FileChosenAlgorithmLoader

SortBench підтримує два способи завантаження алгоритмів сортування - вбудований та зовнішній, кожен з яких реалізований окремим класом-завантажувачем.

BuiltinAlgorithmLoader відповідає за завантаження алгоритмів, що постачаються разом із застосунком. Реалізація є простою та прямолінійною - метод `load()` явно створює екземпляри всіх вбудованих алгоритмів і повертає їх у вигляді списку:

1. BubbleSortAlgorithm;
2. InsertionSortAlgorithm;
3. MergeSortAlgorithm;
4. QuickSortAlgorithm;
5. QuickSortMedianAlgorithm;
6. SelectionSortAlgorithm.

Такий підхід є простим і надійним - додавання нового вбудованого алгоритму вимагає лише одного рядка коду.

Лістинг 2.12 - Клас BuiltinAlgorithmLoader

```
@Slf4j
public class BuiltinAlgorithmLoader {

    public List<SortAlgorithm> load() {
        List<SortAlgorithm> algorithms = new ArrayList<>();
        algorithms.add(new BubbleSortAlgorithm());
        algorithms.add(new InsertionSortAlgorithm());
        algorithms.add(new MergeSortAlgorithm());
        algorithms.add(new QuickSortAlgorithm());
        algorithms.add(new QuickSortMedianAlgorithm());
        algorithms.add(new SelectionSortAlgorithm());

        log.info("Loaded builtin algorithms: {}", algorithms);
    }
}
```

```

        return algorithms;
    }

}

```

`FileChosenAlgorithmLoader` реалізує завантаження зовнішніх алгоритмів, які користувач обирає через файлову систему. Метод `load()` приймає масив обраних файлів і для кожного виконує перевірки: файл має розширення `.class`, а завантажений клас є нащадком `SortAlgorithm`. Якщо перевірки пройдено успішно, створюється екземпляр алгоритму через рефлексію і додається до результату. Файли що не відповідають вимогам ігноруються з відповідним записом у лог. Для завантаження класу з довільного шляху файлової системи використовується `URLClassLoader`, що дозволяє динамічно підключати зовнішній код під час роботи застосунку. [17]

Лістинг 2.13 - Клас `FileChosenAlgorithmLoader`

```

@Slf4j
public class FileChosenAlgorithmLoader {
    private final static String CLASS_EXTENSION = ".class";

    public List<SortAlgorithm> load(File[] files) {
        if (files == null || files.length == 0) {
            return Collections.emptyList();
        }

        List<SortAlgorithm> result = new ArrayList<>();
        for (File file : files) {
            try {
                if (!file.getName().endsWith(CLASS_EXTENSION)) {
                    log.debug("Chosen file by user with name {}
was ignored because the extension of file is not valid", file);
                    continue;
                }

                Class<?> clazz =

```

```

FileUtil.createClass(file.toPath());
        if (!FileUtil.isSubClassOf(clazz,
SortAlgorithm.class)) {
            log.debug("Class with name {} isn't instance
of {}, so it was ignored", clazz, SortAlgorithm.class);
            continue;
        }

        SortAlgorithm sortAlgorithm = (SortAlgorithm)
clazz.getDeclaredConstructor().newInstance();
        result.add(sortAlgorithm);
    } catch (ClassNotFoundException e) {
        log.warn("Not found class {}", file.getName());
    } catch (Exception e) {
        log.warn("Failed to load algorithm from file
{}", file.getName());
    }
}

return result;
}
}

```

Обидва завантажувачі повертають список об'єктів `SortAlgorithm`, що дозволяє `AlgorithmSelectorPanel` відображати вбудовані та зовнішні алгоритми в єдиному списку без жодних відмінностей.

2.5 Розробка бібліотеки `SortBenchAbstraction`

Для реалізації головної функції програми, а саме можливість додавати власні алгоритми сортування, потрібно було зрозуміти, яким чином користувач може реалізувати свій алгоритм не впливаючи на внутрішню логіку застосунку, і щоб при цьому все було доволі просто. Для цієї задачі було використано `UrlClassLoader`, який дозволяє динамічно підвантажувати класи в `Runtime`.

Розробка бібліотеки не створювалась окремо, як це має вигляд зараз. Усі класи, що належать бібліотеці SortBenchAbstraction до цього були частиною всього застосунку. Це не дозволяло створювати алгоритми окремо, щоб імпортувати їх на гарячу, але такий підхід дозволив працювати з робочим прототипом доволі довгий час, який в процесі часто змінювався та покращувався.

Зрештою, всі необхідні абстракції було відокремлено в SortBenchAbstraction. Бібліотека містить мінімальний набір класів, що дозволяє користувачу безперешкодно створити свій власний алгоритм сортування та побачити його роботу в програмі.

SortBenchAbstraction версії 1.0.1 містить два класи:

1. SortAlgorithm - абстрактний клас, що представляє собою алгоритм сортування;
2. SortingCollection - колекція, що зберігає елементи, над якими відбувається сортування.

Лістинг 2.14 - Клас SortAlgorithm

```
public abstract class SortAlgorithm {

    public SortAlgorithm() {}

    public abstract void sort(SortingCollection collection);

    public abstract String getName();

    public abstract void setName(String name);

    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) return
false;
        SortAlgorithm that = (SortAlgorithm) o;

        return getName().equals(that.getName());
    }
}
```



```

@Override
public int hashCode() {
    return Objects.hashCode(getName());
}

@Override
public String toString() {
    return getName();
}
}

```

Будь який алгоритм сортування що наслідує цей клас вміє сортувати та називатись. Алгоритми вважаються однаковими, якщо вони мають однакові назви.

Лістинг 2.15 - Інтерфейс SortingCollection

```

public interface SortingCollection {

    int compare(int index1, int index2);

    Integer peek(int index);

    void swap(int index1, int index2);

    void set(int index, int value);

    int getSize();

}

```

Як видно з лістингу 2.15, така колекція вміє не лише порівнювати та міняти місцями елементи, а й отримувати елемент та встановлювати нові значення. Це було зроблено для того, щоб додати підтримку не класичних алгоритмів сортування, наприклад MergeSort.

SortBenchAbstraction бібліотеку ви можете знайти на Github за посиланням <https://github.com/BohdanYarovyi/Sort-Bench-Abstraction>.

2.6 Публікація бібліотеки на Maven Central Repository

Maven Central Repository - це головний репозиторій усіх бібліотек, що використовуються для розробки на мові Java. Сучасна розробка не обходиться без збірників проектів, на кшталт Maven. Maven та подібні збірники дозволяють розробнику підключати різні залежності до проекту вказавши namespace, назву та версію бібліотеки. Maven Central Repository є місцем, де велика кількість розробників діляться своїм кодом з іншими, щоб інші могли швидко та легко використовувати його у власному коді. [18]

Через те що SortBenchAbstraction є окремою абстракцією, яка має бути єдиною для всіх алгоритмів, було вирішено опублікувати її як бібліотеку у спільному репозиторії Maven, щоб кожен міг підключити її до себе в код і реалізувати свій алгоритм.

Для успішної публікації коду в Central Maven Repository необхідно виконати наступні кроки:

1. Визначити groupId, artifactId, version бібліотеки
2. Налаштувати базовий опис
3. Покрити код повною документацією
4. Вказати розробників
5. Додати ліцензію
6. Створити підписи
7. Опублікувати

Всі кроки були успішно виконані, і тепер SortBenchAbstraction доступна в Maven Repository. [19]

Home » io.github.bohdanyarovi » sort-bench-abstraction



SortBenchAbstraction

A library for creating custom sorting algorithms for use them in SortBenchApplication(<https://github.com/BohdanYarovi/Sort-Bench-Application>).

Overview Versions (2) Used By Badges Books (7)

LICENSE [Apache 2.0](#)

TAGS [github](#) [sorting](#)

RANKING [#271410 in MvnRepository](#)

Homepage <https://github.com/BohdanYarovi/Sort-Bench-Abstraction> [Inspect URL](#)

LINKS 

Latest Versions [2 versions →](#)

Central (2)

	VERSION	VULNERABILITIES	USAGES	DATE
1.0.x	1.0.1		0	May 21, 2026
	1.0		0	May 17, 2026

[2 versions →](#)

Рисунок 2.12 - SortBenchAbstraction на Maven Central Repository

В розділі 3.1 Тестування програмної системи буде більш детально розглянуто, як саме використовувати SortBenchAbstraction для створення власного алгоритму сортування.

3 ТЕСТУВАННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ

3.1 Тестування програмної системи

Невід’ємною частиною розробки є етап тестування. Тестування полягає в тому, щоб перевірити роботу всіх функцій та отримати очікуваний результат.

Тестування сортування

Перший етап тестування полягає у переконанні, що дані сортуються правильно. Для такого тестування запустимо тестування з обраним алгоритмом Bubble Sort.

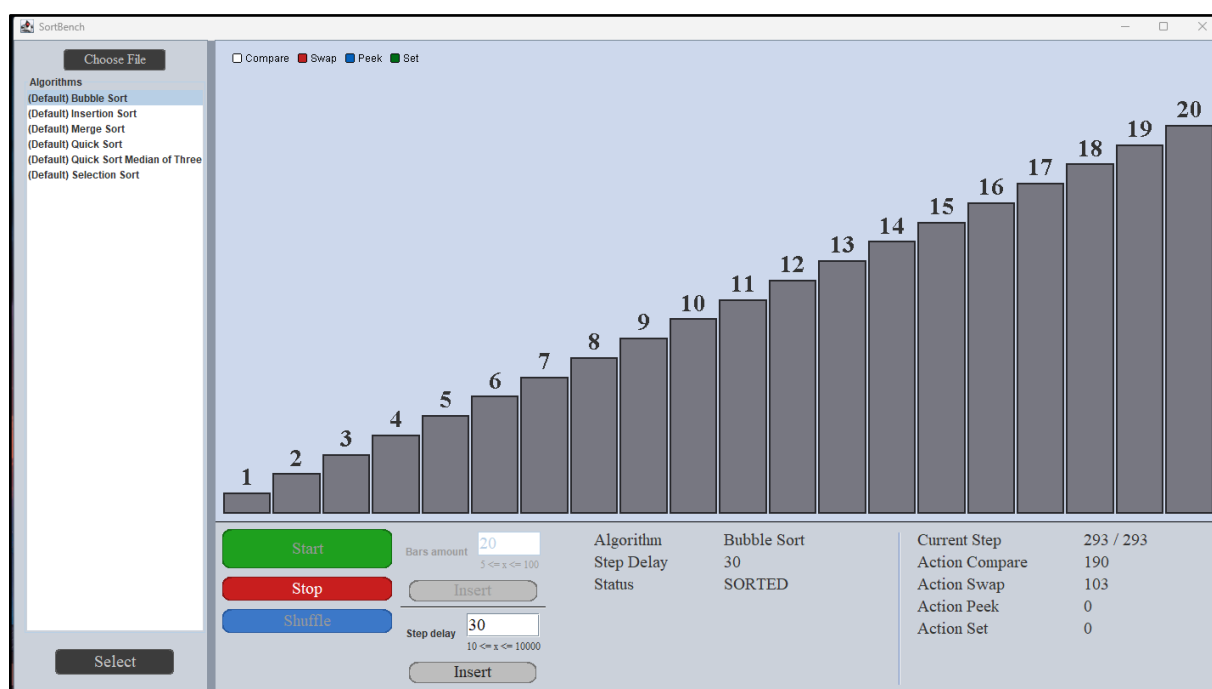


Рисунок 3.1 - Відсортовані дані за алгоритмом Bubble Sort

На рисунку 3.1 зображено правильно посортовані дані з використанням алгоритму Bubble Sort, що були посортовані в 293 кроки. Під час сортування було порівняно 190 елементів між собою та здійснено 103 перестановки.

Тестування завантаження алгоритму

Другим етапом тестування буде перевірка можливості завантаження власного алгоритму сортування. Для цього потрібно реалізувати власний алгоритм сортування з використанням бібліотеки SortBenchAbstraction.

Створимо новий порожній Java проект.

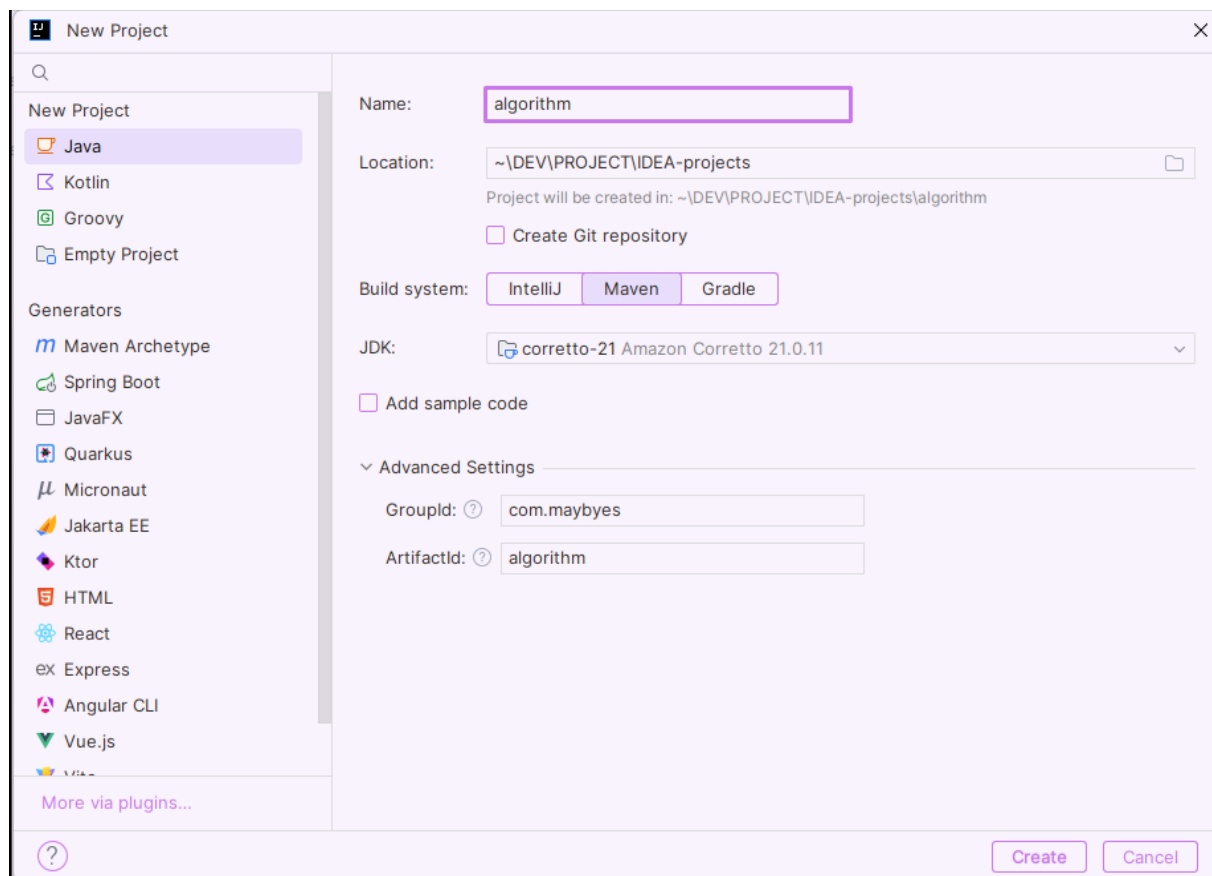


Рисунок 3.2 - Створення нового проекту

Ми будемо реалізовувати Cycle Sort алгоритм. Особливість такого алгоритму полягає в тому, що він дуже оптимально до перестановки елементів. Це корисно в таких випадках, де перестановка елементів дорога для сортування, тому вам потрібно мінімізувати їх.

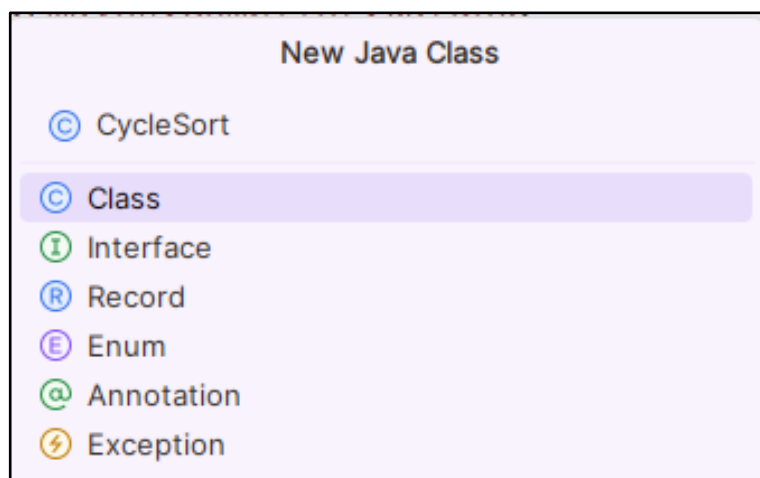


Рисунок 3.3 - Створення класу CycleSort

Далі потрібно підключити бібліотеку `SortBenchAbstraction` та наслідувати клас `CycleSort` від `SortAlgorithm`.

Лістинг 3.1 - Додавання залежності `SortBenchAbstraction`

```
<dependencies>
  <dependency>
    <groupId>io.github.bohdanyarovy</groupId>
    <artifactId>sort-bench-abstraction</artifactId>
    <version>1.0.1</version>
  </dependency>
</dependencies>
```

Лістинг 3.2 - Наслідування `SortAlgorithm`

```
public class CycleSort extends SortAlgorithm {

    @Override
    public void sort(SortingCollection collection) {
        // implement algorithm here
    }

    @Override
    public String getName() {
        // define algorithm name
        return "";
    }

    @Override
    public void setName(String name) {}

}
```

`SortAlgorithm` абстрактний клас вимагає від нас визначення ім'я алгоритму, та визначення методу `sort(SortingCollection collection)`, який безпосередньо сортує дані в `SortingCollection`.

Лістинг 3.3 - Готовий `CycleSort`

```

public class CycleSort extends SortAlgorithm {
    private String name = "CycleSort";

    @Override
    public void sort(SortingCollection collection) {
        int size = collection.getSize();

        for (int cycleStart = 0; cycleStart < size - 1;
cycleStart++) {

            int item = collection.peek(cycleStart);
            int pos = cycleStart;

            // Find the correct position for the current item
            for (int i = cycleStart + 1; i < size; i++) {
                if (collection.peek(i) < item) {
                    pos++;
                }
            }

            // Item already in correct position
            if (pos == cycleStart) {
                continue;
            }

            // Skip duplicates
            while (item == collection.peek(pos)) {
                pos++;
            }

            // Put the item into its correct position
            int temp = collection.peek(pos);
            collection.set(pos, item);
            item = temp;

            // Rotate the rest of the cycle
            while (pos != cycleStart) {
                pos = cycleStart;
            }
        }
    }
}

```

```

        for (int i = cycleStart + 1; i < size; i++) {
            if (collection.peek(i) < item) {
                pos++;
            }
        }

        while (item == collection.peek(pos)) {
            pos++;
        }

        temp = collection.peek(pos);
        collection.set(pos, item);
        item = temp;
    }
}

@Override
public String getName() {
    return name;
}

@Override
public void setName(String name) {
    this.name = name;
}
}

```

Після того, як ми реалізували алгоритм сортування, нам потрібно скомпілювати файл в .class файл. Для цього ми будемо використовувати стандартний компілятор `javac`. Через те що ми використовуємо бібліотеку `SortBenchAbstraction`, нам необхідно вказати шлях до бібліотеки під час компіляції.

Лістинг 3.4 - Компіляція CycleSort файлу


```
javac -cp
C:\Users\Bohdan\.m2\repository\io\github\bohdanyarovy\sort-
bench-abstraction\1.0.1\sort-bench-abstraction-
1.0.1.jar .\src\main\java\CycleSort.java
```

Після компіляції ми отримуємо .class файл, який ми і будемо завантажувати в програму SortBench. Важливо, щоб версія бібліотеки SortBenchAbstraction була сумісна з версією застосунку.

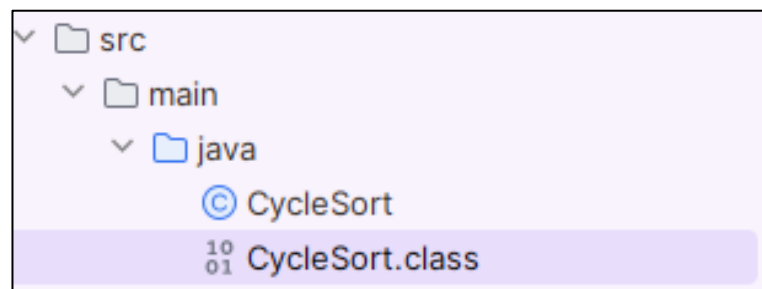


Рисунок 3.4 - Вигляд скомпільованого файлу CycleSort

Тепер ми можемо додати файл CycleSort.class до програми сортування і посортувати елементи алгоритмом сортування Cycle.

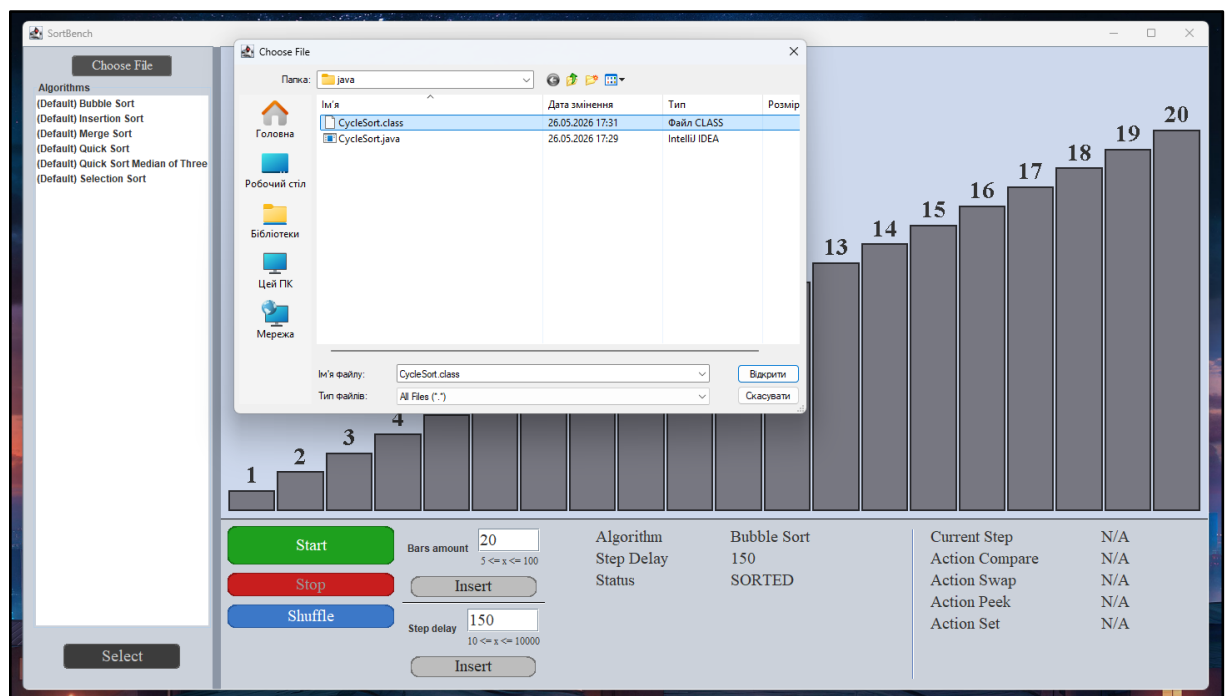


Рисунок 3.5 - Додаємо CycleSort.class

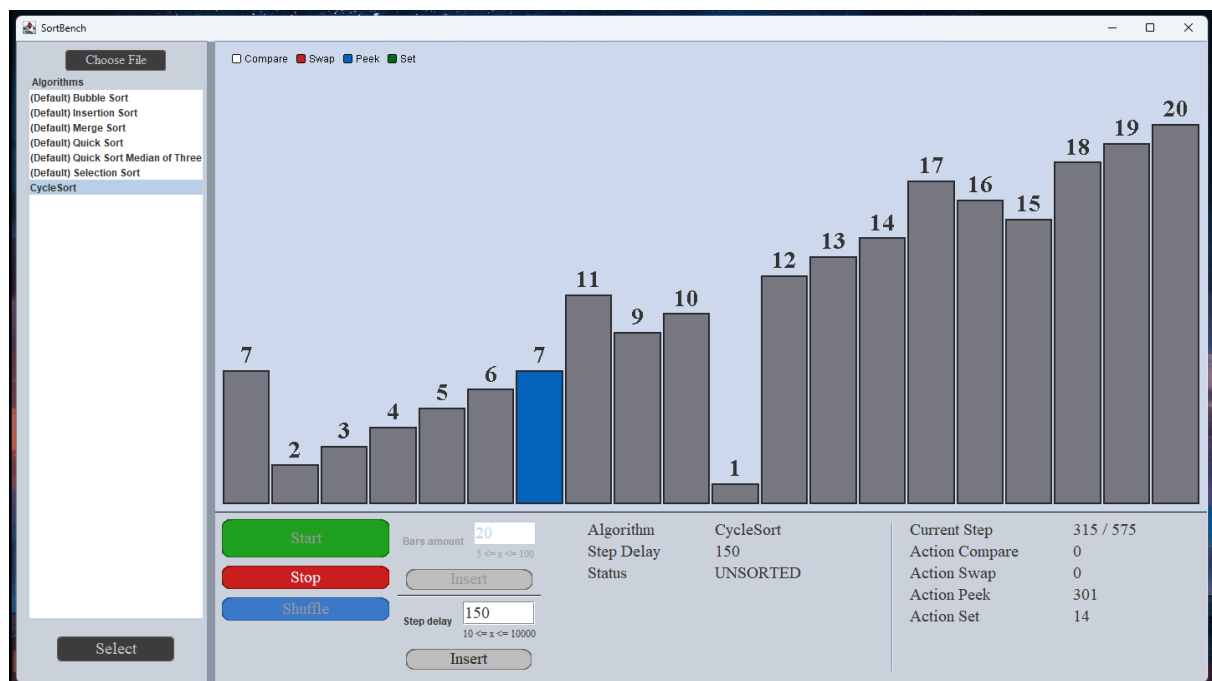


Рисунок 3.6 - Сортування алгоритмом CycleSort

В результаті, елементи були посортовані правильно. При 575 кроках сортування, алгоритм виконав 556 дій Peek та всього 19 дій Set.

Функцію додавання алгоритму сортування було успішно протестовано, та було виявлено, що все працює в штатному режимі. Під час тестування не виникло жодних складнощів.

3.2 Обмеження, недоліки та перспективи розвитку SortBench

SortBench успішно виконує свої основні функції, але все ж вважати це завершеним продуктом не варто. Під час розробки було важливим реалізувати основу, що слугує непоганою платформою для потенційного розширення.

Ось потенційні ідеї для покращення:

1. Додати функціонал для руху між кроками;
2. Додати звуки та анімації;
3. Додати функціонал для визначення вхідних даних. Визначені пресети вхідних даних;
4. Додати метрики використання пам'яті;
5. Додати функціонал для зберігання алгоритмів сортування;

6. Додати функціонал для публічного поширення алгоритмів сортування в інтернеті;
7. Додати функціонал для більш детальної аналітичної звітності щодо роботи алгоритму сортування. Завантаження звітів. Порівняльні таблиці результатів роботи алгоритмів.

Ці функції значно посилять SortBench, та дозволять користувачам розробляти або вивчати алгоритми сортування значно ефективніше.

Серед недоліків варто зазначити:

1. Обмеженість програмного додатку;
2. Додавання алгоритмів лише на мові Java.

ВИСНОВКИ

У результаті виконання кваліфікаційної роботи було розроблено програмну систему SortBench - настільний застосунок для покрокової візуалізації та порівняння алгоритмів сортування засобами Java.

В ході роботи було досліджено теоретичні засади алгоритмів сортування, проведено огляд існуючих інструментів візуалізації та виявлено їх спільний недолік - відсутність зручного механізму для інтеграції власних реалізацій алгоритмів. Саме це стало основою для проєктування архітектури SortBench, де логіка алгоритму повністю відокремлена від логіки відображення завдяки підходу record-and-replay.

Розроблена система підтримує шість вбудованих алгоритмів сортування, надає інструменти для налаштування процесу візуалізації та збору статистики. Ключовою особливістю SortBench є можливість завантаження власних алгоритмів у вигляді скомпільованих `.class` файлів, що стало можливим завдяки розробці та публікації бібліотеки SortBenchAbstraction на Maven Central Repository.

Проведене тестування підтвердило коректність роботи системи. SortBench є готовим інструментом для вивчення та розробки алгоритмів сортування, що має потенціал для подальшого розширення функціоналу.

ПЕРЕЛІК ПОСИЛАНЬ

1. Sorting Algorithms in Computer Science Education [Електронний ресурс]. - Режим доступу: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5088751. - Дата звернення: 20.05.2026.
2. VisuAlgo - visualising data structures and algorithms through animation [Електронний ресурс]. - Режим доступу: <https://visualgo.net/en/sorting>. - Дата звернення: 20.05.2026.
3. Toptal Sorting Algorithms Animations [Електронний ресурс]. - Режим доступу: <https://www.toptal.com/developers/sorting-algorithms>. - Дата звернення: 20.05.2026.
4. Algorithm Visualizer [Електронний ресурс]. - Режим доступу: <https://algorithm-visualizer.org/brute-force/bubble-sort>. - Дата звернення: 20.05.2026.
5. The Sound of Sorting [Електронний ресурс]. - Режим доступу: <https://panthema.net/2013/sound-of-sorting/>. - Дата звернення: 20.05.2026.
6. USFCA Data Structure Visualizations [Електронний ресурс]. - Режим доступу: <https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>. - Дата звернення: 20.05.2026.
7. Дані [Електронний ресурс] // Вікіпедія. - Режим доступу: <https://uk.wikipedia.org/wiki/Дані>. - Дата звернення: 20.05.2026.
8. Таблиця Unicode [Електронний ресурс]. - Режим доступу: <https://xn--80adth0aefm3i.xn--j1amh/таблиця-unicode>. - Дата звернення: 20.05.2026.
9. У чому полягає сортування даних: принципи та алгоритми [Електронний ресурс]. - Режим доступу: <https://www.newsukraine.com.ua/ru/u-chomu-polyagaye-sortuvannya-danyh-pryncypy-ta-algorytmy/>. - Дата звернення: 20.05.2026.
10. Алгоритм сортування [Електронний ресурс] // Вікіпедія. - Режим доступу: https://uk.wikipedia.org/wiki/Алгоритм_сортування. - Дата звернення: 20.05.2026.

11. Алгоритми сортування, злиття та пошуку [Електронний ресурс]. - Режим доступу: https://e-tk.lntu.edu.ua/pluginfile.php/20057/mod_resource/content/0/Тема%203.%20Алгоритми%20сортування,%20злиття%20та%20пошуку.pdf. - Дата звернення: 20.05.2026.
12. QuickSort: історія виникнення та розвитку найшвидшого алгоритму сортування [Електронний ресурс]. - Режим доступу: <https://phm.cuspu.edu.ua/nauka/naukovo-populiarni-publikatsii/824-quicksort-istoriia-vynyknennia-ta-rozvytku-naishvydshoho-alhorytmu-sortuvannia.html>. - Дата звернення: 20.05.2026.
13. Van Wyk C. The one perfect sorting algorithm [Електронний ресурс]. - Режим доступу: <https://www.iljitsch.com/2020/11-22-the-one-perfect-sorting-algorithm.html>. - Дата звернення: 20.05.2026.
14. Sedgewick R., Wayne K. Algorithms. - 4th ed. - Addison-Wesley, 2011. - 955 p.
15. Яровий Б. С. SortBench [Електронний ресурс]. - 2026. - Режим доступу: <https://github.com/BohdanYarovyi/Sort-Bench-Application>. - Дата звернення: 20.05.2026.
16. Oracle Corporation. Java SE Swing API Documentation [Електронний ресурс]. - Режим доступу: <https://docs.oracle.com/javase/8/docs/api/javax/swing/package-summary.html>. - Дата звернення: 20.05.2026.
17. Flanagan D. Java in a Nutshell. - 5th ed. - O'Reilly Media, 2005. - 928 p.
18. Sonatype. Maven Central Repository [Електронний ресурс]. - Режим доступу: <https://central.sonatype.com/>. - Дата звернення: 20.05.2026.
19. Яровий Б. С. SortBenchAbstraction [Електронний ресурс]. - 2026. - Режим доступу: <https://central.sonatype.com/artifact/io.github.bohdanyarovyi/sort-bench-abstraction>. - Дата звернення: 20.05.2026.

ДОДАТКИ

ДОДАТОК А

Повний вихідний код програмної системи SortBench доступний у публічному репозиторії GitHub за посиланням: <https://github.com/BohdanYarovyi/Sort-Bench-Application>

Вихідний код бібліотеки SortBenchAbstraction доступний за посиланням: <https://github.com/BohdanYarovyi/Sort-Bench-Abstraction>

ПРЕЗЕНТАЦІЯ

2026

Розробка програмної системи для візуалізації, порівняння та аналізу ефективності алгоритмів сортування засобами Java

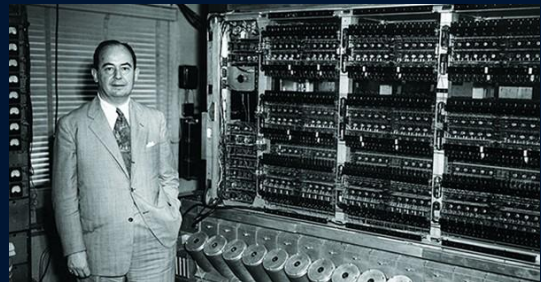
Комп'ютерні науки

Виконав - Яровий Богдан, КНД -42
Керівник - Березовська Юлія Володимирівна

Сортування даних

«Сортування та пошук — це серцебиття комп'ютера. Усе інше просто перекачує кров з місця на місце»

Дональд Ервин Кнут



У 1945 році Джон фон Нейман створив першу в історії програму для ЕОМ, і її суттю було сортування великих об'ємів даних.

Дві причини, чому сортування важливе для інтелектуальних технологій



Прискорення пошуку інформації

Швидко знайти потрібний елемент у несортованому масиві можна лише повним перебором, що займає багато часу. У сортованих даних можна використовувати ефективні алгоритми, наприклад, бінарний пошук, який знаходить інформацію майже миттєво навіть серед мільярдів записів

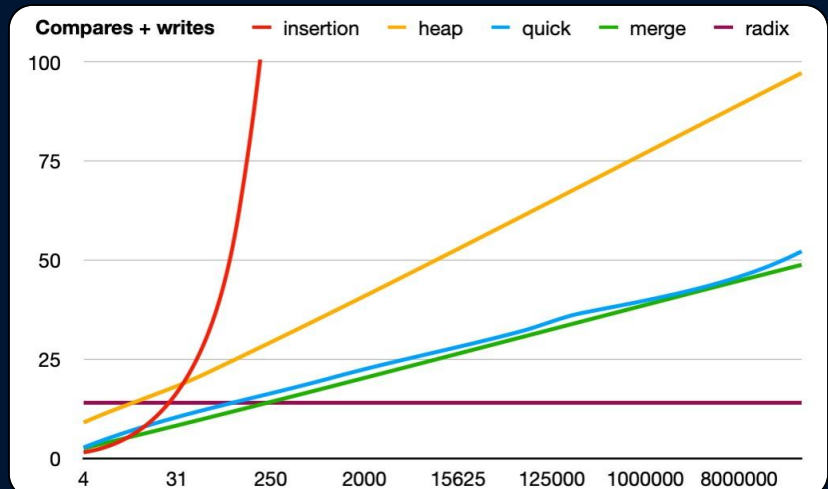


Спрощення аналізу та візуалізації

Впорядковані дані набагато легше аналізувати як людині, так і комп'ютерним програмам. Сортування дозволяє швидко виявляти екстремальні значення (мінімуми та максимуми), групувати схожі елементи, знаходити дублікати, вираховувати медіану та будувати зрозумілі графіки чи звіти.

Найвідоміші алгоритми сортування

- Insertion Sort
або
Bubble Sort
- Heap Sort
- Quick Sort
- Merge Sort
- Radix Sort



В чому проблема?

Алгоритми сортування використовується в 99.9% усіх систем у світі.

Знання таких алгоритмів є **must have** для кожного розробника в ІТ.

Зрозуміти такі алгоритми **складно** - бо вони складні та абстрактні.

Що робити?

Краще один раз **побачити** ніж сто разів почути.



Існує безліч рішень, але є нюанс..

В інтернеті вистачає різноманітних ілюстраторів алгоритмів сортування. Від приватних рішень на GitHub, до публічних web-сайтів.

Головна проблема таких рішень - алгоритми жорстко заковані. **Не можна додати свій власний алгоритм.**

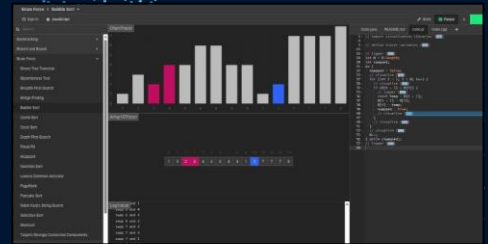


Виняток чи exception?

Algorithm Visualizer - web-ресурс для демонстрації роботи алгоритмів.

Його перевага перед подібними ресурсами в тому, що він надає можливість програмувати власні алгоритми, і не тільки алгоритми сортування. Писати код можна на мовах: Java, JavaScript, C++.

Але не все так однозначно ...



```
1 import org.algorithm_visualizer.*;
2
3 import java.util.Arrays;
4
5 public class Main {
6     private static ChartTracer chartTracer = new ChartTracer();
7     private static LogTracer logTracer = new LogTracer("Console");
8
9     private static Integer [] array = (Integer[]) new Randomize.ArrayID(15, new Randomize.Integer(1, 20)).create();
10
11     public static void main(String[] args) {
12         int length = array.length;
13
14         logTracer.printf("original array = %s\n", Arrays.toString(array));
15         chartTracer.set(array);
16         Layout.setRoot(new VerticalLayout(new Commander[]{chartTracer, logTracer}));
17         Tracer.delay();
18
19         boolean flag;
20
21         for (int i = length - 1; i > 0; i--) {
22             flag = true;
23             for (int j = 0; j < i; j++) {
24                 chartTracer.select(j);
25                 chartTracer.select(j + 1);
26                 if (array[j] > array[j + 1]) {
27                     logTracer.printf("swap %s and %s\n", array[j], array[j + 1]);
28                     swap(j, j + 1, array);
29                     flag = false;
30                 }
31                 chartTracer.deselect(j);
32                 chartTracer.deselect(j + 1);
33             }
34             if (flag) {
35                 break;
36             }
37         }
38
39         logTracer.printf("sorted array = %s\n", Arrays.toString(array));
40
41         private static void swap(int x, int y, Integer [] array) {
42             //
43         }
44     }
45 }
```

Якщо подивитись ближче, то можна зрозуміти, що код робить все:

- бізнес-логіку;
- генерацію даних;
- відображення елементів;
- логування.

```
1 import org.algorithm_visualizer.*;
2
3 import java.util.Arrays;
4
5 public class Main {
6     private static ChartTracer chartTracer = new ChartTracer();
7     private static LogTracer logTracer = new LogTracer("Console");
8
9     private static Integer [] array = (Integer[]) new Randomize.ArrayID(15, new Randomize.Integer(1, 20)).create();
10
11     public static void main(String[] args) {
12         int length = array.length;
13
14         logTracer.printf("original array = %s\n", Arrays.toString(array));
15         chartTracer.set(array);
16         Layout.setRoot(new VerticalLayout(new Commander[]{chartTracer, logTracer}));
17         Tracer.delay();
18
19         boolean flag;
20
21         for (int i = length - 1; i > 0; i--) {
22             flag = true;
23             for (int j = 0; j < i; j++) {
24                 chartTracer.select(j);
25                 chartTracer.select(j + 1);
26                 if (array[j] > array[j + 1]) {
27                     logTracer.printf("swap %s and %s\n", array[j], array[j + 1]);
28                     swap(j, j + 1, array);
29                     flag = false;
30                 }
31                 chartTracer.deselect(j);
32                 chartTracer.deselect(j + 1);
33             }
34             if (flag) {
35                 break;
36             }
37         }
38
39         logTracer.printf("sorted array = %s\n", Arrays.toString(array));
40
41         private static void swap(int x, int y, Integer [] array) {
42             //
43         }
44     }
45 }
```

SORT BENCH

Застосунок - який може



Sort Bench - це програмний засіб, який створений для візуалізації, аналізу та розробки алгоритмів сортування.

Головний принцип - просто та зрозуміло.

Головні функції



1. Відображення та візуалізація переміщення елементів колекції
2. Вибір алгоритмів сортування
3. Налаштування умов сортування
4. Можливість додати власний алгоритм
5. Метрики операцій

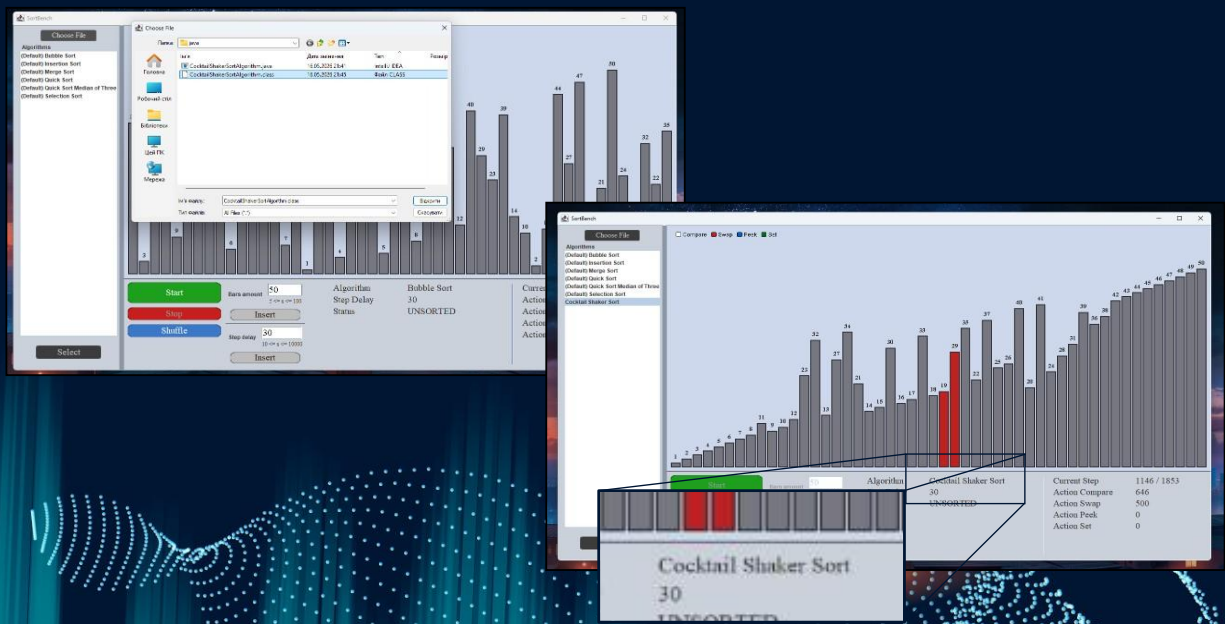
Record-and-replay підхід

Для відображення роботи алгоритму сортування було впроваджено record-and-replay підхід:

- Користувач натискає кнопку **Start**
- Елементи передаються алгоритму сортування, де кожна взаємодія з колекцією елементів зберігається
- Вікно з елементами повторює події



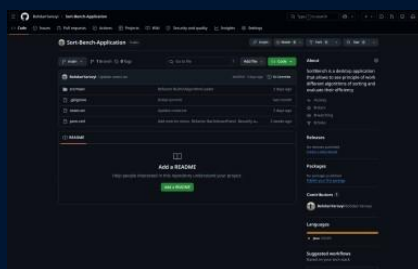
Додавання власного алгоритму



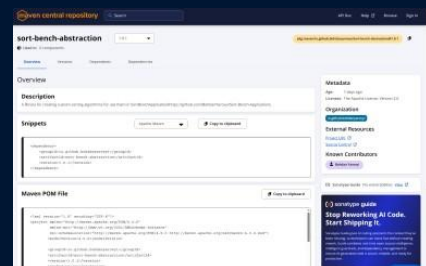
Sort Bench Abstraction

Sort-Bench-Abstraction - це легковісна бібліотека, яка потрібна для створення алгоритмів, що можна використовувати в Sort Bench.

Бібліотека доступна на таких платформах

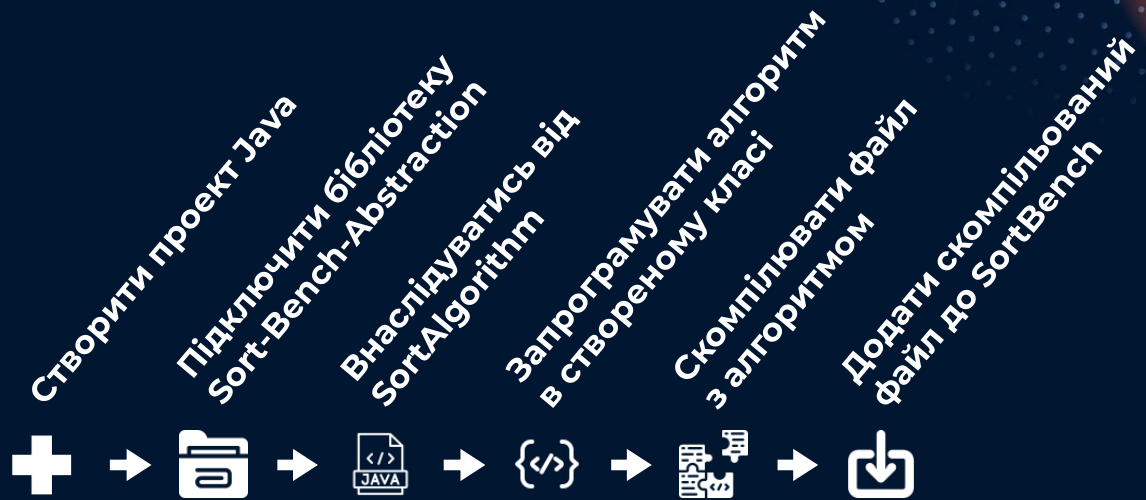


GitHub



Maven Central Repository

Roadmap по запуску свого алгоритму сортування в Sort Bench



Код алгоритму Bubble Sort

Sort Bench

```
public class BubbleSortAlgorithm extends SortAlgorithm {
    private String name = "Bubble Sort";

    @Override
    public void sort(SortingCollection collection) {
        int size = collection.getSize();
        for (int i = 0; i < size-1; i++) {
            for (int j = 0; j < size-1-i; j++) {
                if (collection.compare(j, j+1) > 0) {
                    collection.swap(j, j+1);
                }
            }
        }
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public void setName(String name) {
        this.name = name;
    }
}
```

Algorithm Visualizer

```
public class Main {
    private static ChartTracer chartTracer = new ChartTracer();
    private static LogTracer logTracer = new LogTracer("Console");

    private static Integer[] array = (Integer[]) new RandomizeArray(0)(5, new RandomizeInteger(0, 20)).create();

    public static void main(String[] args) {
        int length = array.length;

        logTracer.print("original array = " + array.toString());
        chartTracer.setArray(array);
        Layout.setRoot(new VerticalLayout(new Commander[] {chartTracer, logTracer}));
        Tracer.delay();

        boolean flag;

        for (int i = length-1; i > 0; i--) {
            flag = true;
            for (int j = 0; j < i; j++) {
                chartTracer.select(j);
                chartTracer.select(j+1);
                Tracer.delay();
                if (array[j] > array[j+1]) {
                    logTracer.print("swap " + j + " and " + (j+1) + ": " + array[j] + " and " + array[j+1]);
                    swap(j, j+1, array);
                    flag = false;
                }
                chartTracer.deselect(j);
                chartTracer.deselect(j+1);
            }
            if (flag) {
                break;
            }
        }

        logTracer.print("sorted array = " + array.toString());
    }

    private static void swap(int x, int y, Integer[] array) {
        int temp = array[x];
        array[x] = array[y];
        array[y] = temp;
        chartTracer.patch(x, array[x]);
        chartTracer.patch(y, array[y]);
        Tracer.delay();
        chartTracer.depatch(x);
        chartTracer.depatch(y);
    }
}
```

Висновки

- Розроблено програмну систему SortBench для візуалізації алгоритмів сортування
- Реалізовано 6 алгоритмів сортування з покроковою візуалізацією
- Розроблено та опубліковано бібліотеку SortBenchAbstraction на Maven Central
- Реалізовано механізм завантаження власних алгоритмів без модифікації застосунку
- Проведено тестування — всі функції працюють коректно

